Mini Project Report On

Dynamic Resource Aggregation and Allocation Across Multiple Android Devices for a Distributed Personal Cloud

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

By

David George (U2303080)

Joel Sony (U2303118)

Joseph Thankachan (U2303121)

Kevin Jose (U2303131)

Under the guidance of

Ms. Seema Safar

Department of Computer Science & Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Affiliated to APJ Abdul Kalam Technological University)
Rajagiri Valley, Kakkanad, Kochi, 682039

March 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
RAJAGIRI SCHOOL OF ENGINEERING & TECHNOLOGY (AUTONOMOUS)
RAJAGIRI VALLEY, KAKKANAD, KOCHI, 682039
(Affiliated to APJ Abdul Kalam Technological University)

CERTIFICATE

This is to certify that the mini project report entitled ***"Dynamic Resource Aggregation and Allocation Across Multiple Android Devices for a Distributed Personal Cloud"*** *is a bonafide record of the work done by* ***David George (U2303080)***, ***Joel Sony (U2303118)***, ***Joseph Thankachan (U2303121)***, ***Kevin Jose (U2303131)***, *submitted to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in Computer Science and Engineering during the academic year 2025-2026.*

Ms. Seema Safar
Mini-Project Guide
Asst. Professor
Dept. of CSE
RSET

Ms. Seema Safar
Mini-Project Coordinator
Asst. Professor
Dept. of CSE
RSET

Dr. Preetha K.G.
Head of Department
Dept. of CSE
RSET

ACKNOWLEDGEMENTS

I wish to express my sincere gratitude towards Rev. Dr. Jaison Paul Mulerikkal CMI, Principal of RSET, and Dr. Preetha K.G., Head of the Department of Computer Science and Engineering for providing me with the opportunity to undertake my mini project, "Dynamic Resource Aggregation and Allocation Across Multiple Android Devices for a Distributed Personal Cloud".

I am highly indebted to my project coordinators, **Ms. Seema Safar**, Asst. Professor, Department of Computer Science and Engineering for their valuable support.

It is indeed my pleasure and a moment of satisfaction for me to express my sincere gratitude to my project guide **Ms. Seema Safar** for her patience and all the priceless advice and wisdom she has shared with me.

Last but not the least, I would like to express my sincere gratitude towards **Ms. Sherine Sebastian** and all other teachers and friends for their continuous support and constructive ideas.

David George

Joel Sony

Joseph Thankachan

Kevin Jose

Abstract

Traditional cloud storage systems rely heavily on centralized infrastructure managed by third-party providers, which raises concerns regarding data privacy, trust, and control. This project proposes a decentralized personal mobile cloud storage system that utilizes user-owned smartphones as distributed storage nodes. The system consists of an Android application operating in two modes—User Mode and Storage Node Mode—and a lightweight coordination server responsible for metadata management and device orchestration.

In the proposed system, files are encrypted on the user device before transmission and divided into multiple chunks. These encrypted chunks are then distributed and replicated across selected storage nodes within the user's personal cluster of mobile devices. The central server coordinates chunk placement, monitors node availability using heartbeat signals, and maintains metadata such as chunk mappings and device status, while never accessing plaintext data. Secure communication is maintained through Representational State Transfer (REST) APIs and WebSocket connections, ensuring efficient coordination between devices.

Unlike traditional cloud storage solutions that rely on external providers, this approach removes the need for third-party trust by implementing a zero-trust architecture, where both the storage nodes and the coordination server cannot access or infer the contents of stored files. The system also incorporates data integrity verification using cryptographic hashing and client-side encryption to ensure confidentiality and reliability.

The proposed architecture demonstrates how modern smartphones can be leveraged to create a user-managed, decentralized storage environment, offering improved privacy, enhanced control over data, and reduced dependency on centralized cloud providers.

Contents

Acknowledgements **i**

Abstract **ii**

List of Figures **vi**

List of Tables **vii**

List of Abbreviations **viii**

1 Introduction **1**

1.1 Background 1

1.2 Problem Definition 2

1.3 Scope and Motivation 2

1.4 Objectives 3

1.5 Challenges 3

1.6 Assumptions 4

1.7 Societal / Industrial Relevance 4

1.8 Organization of the Report 4

2 Software Requirements Specification **5**

2.1 Introduction 5

2.2 Overall Description 5

2.3 External Interface Requirements 6

2.3.1 User Interface 6

2.3.2 Hardware Interface 6

2.3.3 Software Interface 6

2.3.4 Communication Interface 7

2.4 System Features 7

2.4.1 Device Registration 7
2.4.2 File Upload and Chunking 7
2.4.3 Dynamic Resource Allocation 7
2.4.4 Chunk Replication 7
2.4.5 File Retrieval 7
2.4.6 Device Monitoring 8
2.5 Other Nonfunctional Requirements 8
2.5.1 Security 8
2.5.2 Performance 8
2.5.3 Reliability 8
2.5.4 Scalability 8

3 System Architecture and Design 9
3.1 System Overview 9
3.2 Architectural Design 10
3.2.1 Use Case Diagram 11
3.2.2 Sequence Diagram 12
3.2.3 System Architecture 14
3.3 Dataset identified 15
3.4 Proposed Methodology and Algorithms 15
3.5 User Interface Design 20
3.6 Database Design 26
3.7 Description of Implementation Strategies 28
3.7.1 Android Application 29
3.7.2 Backend Server 29
3.7.3 REST API Communication 29
3.7.4 File Encryption Using ASCON 30
3.7.5 File Chunking and Distribution 30
3.7.6 Semantic Search Using Embedding Models 30
3.7.7 Distributed Storage Coordination 31
3.8 Module Division 31
3.8.1 Android App Module 31

3.8.2 Backend Module 32
3.8.3 WebSocket Module 32
3.8.4 Services Module 32
3.8.5 Data Models Module 33
3.8.6 Utilities Module 33
3.8.7 Semantic Search Module 33
3.8.8 Core Infrastructure Module 33
3.8.9 Foreground Service Module 34
3.8.10 File Management Module 34
3.9 Work Schedule-Gantt Chart 34

4 Results and Discussions **36**
4.1 Overview of Results 36
4.2 Testing 36
4.3 Quantitative Results 38
4.4 Graphical Analysis 38
4.5 Discussion 40

5 Conclusion **41**
5.1 Conclusion 41
5.2 Future Scope 42

References **43**

Appendix A: Presentation **44**

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes **66**

Vision, Mission, POs, PSOs and COs **ii**

Appendix C: CO-PO-PSO Mapping **vi**

List of Figures

3.1 Use Case Diagram of the Proposed System 11
3.2 Sequence Diagram for File Upload Process 12
3.3 Sequence Diagram for File Retrieval Process 13
3.4 System Architecture of the Distributed Personal Cloud 14
3.5 Mode Selection Interface 20
3.6 Storage Status Interface 21
3.7 Storage Node Interface 22
3.8 File Upload Interface 23
3.9 Browse Files Interface 24
3.10 Semantic Search and File Download Interface 25
3.11 Project Work Schedule Gantt Chart 35

4.1 Administrative interface displaying device status and replication health in the storage cluster. 37
4.2 Relationship between chunk size and upload latency in the distributed storage system. 39
4.3 Latency breakdown of different stages involved in the semantic search pipeline. 40

List of Tables

3.1 Structure of the Users Table 26
3.2 Structure of the Files Table 27
3.3 Structure of the Devices Table 27
3.4 Structure of the Chunks Table 28
3.5 Structure of the Chunk Replication Table 28

List of Abbreviations

REST	Representational State Transfer
API	Application Programming Interface
ASCON	Authenticated Lightweight Cryptographic Algorithm
AEAD	Authenticated Encryption with Associated Data
ORM	Object Relational Mapping
WS	WebSocket
TF-IDF	Term Frequency–Inverse Document Frequency
ONNX	Open Neural Network Exchange

Chapter 1

Introduction

1.1 Background

Cloud storage services such as Google Drive, Dropbox, and OneDrive are widely used for storing and accessing digital data from anywhere. These systems rely on centralized servers managed by third-party providers, offering convenient storage, synchronization, and scalability. However, centralized cloud storage raises concerns regarding data privacy, security, and user control, since users must trust external providers to store and manage their personal data. Risks such as data breaches, unauthorized access, and misuse of stored information have made privacy an important issue in modern cloud systems.

Another limitation of traditional cloud storage is the strong dependence on external infrastructure. Users rely entirely on service providers for maintaining availability and managing data. Even when files are encrypted before uploading, they still remain dependent on centralized platforms for storage and retrieval.

At the same time, modern smartphones possess significant storage capacity, computing power, and reliable network connectivity. Many users also own multiple devices that often remain underutilized. Additionally, a large number of older smartphones are discarded every year, contributing to the growing problem of electronic waste (e-waste). Many of these devices remain functional but are no longer actively used for everyday tasks. Utilizing such devices as part of a distributed storage network can help extend their useful lifetime and reduce electronic waste.

This project proposes a decentralized personal mobile cloud storage system where user-owned smartphones act as storage nodes. Files are encrypted on the user device, divided into chunks, and distributed across multiple devices. A lightweight coordination server manages metadata and device availability without accessing the actual data. This approach enhances data privacy, user control, and reliability while also promoting

sustainable use of existing hardware resources.

1.2 Problem Definition

Traditional cloud storage systems rely on centralized servers managed by third-party providers, which raises concerns regarding data privacy, security, and user control. Users must depend on external infrastructure to store and manage their personal data, creating risks such as unauthorized access, data breaches, and service outages. At the same time, many smartphones possess unused storage and computing resources that remain underutilized or are discarded despite being functional. The problem addressed in this project is the design and implementation of a secure decentralized storage system that utilizes clustered mobile phones as distributed storage nodes. The system must ensure data confidentiality through client-side encryption, reliable storage through chunking and replication mechanisms, and effective monitoring of device availability while maintaining acceptable performance..

1.3 Scope and Motivation

The scope of this project is to design and implement a decentralized personal mobile cloud storage system that utilizes smartphones as distributed storage nodes. The system allows users to securely store files across multiple user-owned mobile devices instead of relying on centralized cloud providers. A lightweight coordination server manages metadata, device registration, chunk mapping, and node availability without accessing the actual file contents. Files are encrypted on the user device, divided into chunks, and replicated across multiple storage nodes to ensure reliability and availability. The project mainly focuses on secure file storage, device coordination, chunk replication, and retrieval using Android devices and a FastAPI-based backend server.

The motivation for this project arises from increasing concerns regarding data privacy, security, and dependency on third-party cloud providers. Traditional cloud storage systems require users to trust external organizations with their personal data, which may expose sensitive information to security risks. At the same time, modern smartphones possess significant computing power and storage capacity that often remains underutilized. Additionally, a large number of older smartphones are discarded every year, contributing

to the growing problem of electronic waste (e-waste). By utilizing such devices as part of a distributed storage network, their useful lifetime can be extended while reducing electronic waste. This project aims to explore how decentralized storage using mobile devices can improve privacy, data ownership, and control over personal data while promoting more sustainable use of existing hardware resources.

1.4 Objectives

- To design and implement a decentralized mobile cloud storage system that utilizes user-owned smartphones as storage nodes.
- To ensure data security and privacy by implementing client-side encryption so that files are encrypted before leaving the user device.
- To develop a coordination server that manages metadata, device registration, chunk mapping, and node availability without accessing actual file contents.
- To implement chunk-based file storage and replication across multiple storage nodes to improve reliability and fault tolerance.
- To enable secure communication between devices and the server using Representational State Transfer (REST) Application Programming Interface (API)s and WebSocket (WS) connections.
- To provide a user-friendly Android interface that allows users to upload, retrieve, and manage files in a cloud-like environment.

1.5 Challenges

Developing a decentralized mobile storage system involves challenges such as managing device availability, ensuring reliable data replication, and maintaining secure communication between multiple devices. Since storage nodes are mobile phones that may frequently go offline, the system must continuously monitor node status and handle failures during file storage and retrieval. Another challenge is implementing strong encryption and integrity verification while coordinating efficient chunk distribution and reconstruction across distributed devices.

1.6 Assumptions

The project assumes that the mobile devices participating in the storage cluster have stable internet connectivity to communicate with the coordination server. It is also assumed that the smartphones used as storage nodes are owned and managed by the user, ensuring trust within the cluster environment. Additionally, the system assumes that the coordination server remains available so that devices can register, send heartbeat signals, and retrieve metadata required for storage and file retrieval operations.

1.7 Societal / Industrial Relevance

The proposed decentralized mobile cloud storage system can be applied in environments where users require secure and private data storage without depending on third-party cloud providers. The system is particularly suitable for small-scale deployments, such as in homes or small groups, where users can utilize their own smartphones as storage nodes to create a personal storage cluster. This allows users to maintain full control over their data while reducing reliance on external cloud services.

The project is relevant in scenarios where privacy, data ownership, and cost efficiency are important. Additionally, the project demonstrates how underutilized smartphone storage and computing resources can be leveraged to build decentralized storage systems. This approach promotes user-controlled data management and contributes to research areas such as mobile cloud computing, distributed systems, and edge computing.

1.8 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 presents the Software Requirements Specification (SRS) of the proposed system, including the overall description, external interface requirements, system features, and other non-functional requirements.

Chapter 3 describes the system architecture and design of the proposed decentralized mobile storage system. It explains the system overview, architectural design, module division, proposed methodology, user interface design, database design, implementation strategies, and the work schedule represented using a Gantt chart.

Chapter 2

Software Requirements Specification

2.1 Introduction

This Software Requirements Specification (SRS) document describes the requirements for the system titled **Dynamic Resource Aggregation and Allocation Across Multiple Android Devices for a Distributed Personal Cloud**. The purpose of this document is to define the functional and non-functional requirements of the proposed system and to serve as a reference for developers, evaluators, and stakeholders during the development process.

The proposed system enables users to create a personal cloud storage environment using multiple Android devices owned by the user. Instead of relying on centralized cloud storage providers, the system distributes encrypted file chunks across user-controlled devices. A central coordination server manages metadata, device registration, and communication between devices without accessing the actual data stored within the system.

The document provides an overview of the system architecture, system features, interface requirements, and operational constraints required for the successful implementation of the system.

2.2 Overall Description

The proposed system is designed as a decentralized personal cloud storage solution that dynamically aggregates and allocates storage resources across multiple Android devices. The system allows users to use their own smartphones as storage nodes, thereby forming a distributed storage cluster.

The system consists of two major components: an Android application and a central coordination server. The Android application operates in two modes: *User Mode* and *Storage Node Mode*. In User Mode, users can upload, download, browse, and search files

stored in the cluster. In Storage Node Mode, the device contributes its storage resources to the cluster by storing encrypted file chunks.

The central server maintains metadata about users, devices, files, and chunk mappings. It also monitors the availability of cluster devices using periodic heartbeat signals and coordinates the distribution and replication of file chunks across the cluster. To ensure security and privacy, files are encrypted on the user device before transmission, and only encrypted chunks are stored on storage nodes.

The system aims to provide a self-hosted, decentralized cloud environment where users have complete control over their data without relying on third-party cloud providers.

2.3 External Interface Requirements

The system interacts with users, hardware components, and software modules through well-defined interfaces.

2.3.1 User Interface

The Android application provides a graphical user interface through which users can interact with the system. The interface allows users to select operating modes, upload files, browse stored files, perform searches, and monitor the status of storage nodes in the cluster.

2.3.2 Hardware Interface

The system operates on Android smartphones which provide the required hardware resources such as storage, processor capabilities, and network connectivity. These devices act either as user devices or as storage nodes in the cluster.

2.3.3 Software Interface

The system communicates with a backend server through REST APIs and WebSocket connections. The server is responsible for managing metadata, coordinating device communication, and maintaining system status.

2.3.4 Communication Interface

Communication between devices and the server occurs over secure internet connections. REST-based APIs are used for registration and file upload operations, while WebSocket connections enable real-time communication for commands, heartbeats, and cluster coordination.

2.4 System Features

The system provides several features that enable distributed storage and resource management across multiple Android devices.

2.4.1 Device Registration

Android devices can register with the central server by providing identification details such as device fingerprint, device name, and storage capacity. The server assigns a unique device identifier and stores the device information in the metadata database.

2.4.2 File Upload and Chunking

When a user uploads a file, the system encrypts the file and divides it into smaller chunks. These chunks are then transmitted to the server for distribution across available storage nodes.

2.4.3 Dynamic Resource Allocation

The server dynamically selects storage nodes based on their available storage capacity and online status. This ensures efficient utilization of resources within the cluster.

2.4.4 Chunk Replication

To ensure data reliability and fault tolerance, multiple replicas of each file chunk are stored across different devices. This prevents data loss if a device becomes unavailable.

2.4.5 File Retrieval

When a user requests a file, the server collects the required chunks from active storage nodes, reconstructs the original file, and sends it back to the user.

2.4.6 Device Monitoring

The system monitors storage nodes using periodic heartbeat signals. Devices that fail to respond within a specified time interval are marked as offline.

2.5 Other Nonfunctional Requirements

The system must satisfy several non-functional requirements to ensure performance, reliability, and security.

2.5.1 Security

All files must be encrypted on the user device before transmission. Encryption keys remain on the user device, and the server handles only metadata without accessing the actual file contents.

2.5.2 Performance

The system should handle file uploads, downloads, and chunk distribution efficiently while maintaining stable communication between devices.

2.5.3 Reliability

The system must maintain multiple replicas of file chunks to ensure that data remains available even if some storage nodes become unavailable.

2.5.4 Scalability

The system should support the addition of new storage nodes, allowing the total storage capacity of the cluster to increase dynamically.

Chapter 3

System Architecture and Design

3.1 System Overview

The proposed system aims to create a decentralized personal cloud storage environment by utilizing the storage resources available across multiple Android devices owned by a user. Unlike traditional cloud storage systems that depend on centralized third-party providers, this system aggregates storage capacity from user-owned mobile devices and forms a distributed storage cluster capable of securely storing and retrieving files.

The architecture of the system consists of two major components: an Android application running on user devices and a central coordination server that manages communication and metadata. The Android application acts as the primary interface through which users interact with the system. It allows users to upload, download, search, and manage files stored within the distributed personal cloud.

Each Android device participating in the system can operate in two different modes: *User Mode* and *Storage Node Mode*. In User Mode, the device acts as a client that allows the user to access files stored in the distributed storage cluster. In Storage Node Mode, the device contributes a portion of its local storage to the system and stores encrypted file chunks received from the coordination server.

When a user uploads a file through the Android application, the file is first encrypted locally on the user device to ensure data confidentiality. After encryption, the file is divided into smaller fixed-size chunks. These chunks are then transmitted to the central coordination server. The server analyzes the available storage nodes in the network and dynamically selects suitable devices based on their available storage capacity, network availability, and operational status.

Once appropriate storage nodes are selected, the server distributes the encrypted chunks across multiple devices in the cluster. To improve system reliability and fault

tolerance, the system creates multiple replicas of each chunk and stores them on different devices. This ensures that the file can still be reconstructed even if one or more storage nodes become unavailable.

The coordination server continuously monitors the status of connected devices through periodic heartbeat signals sent by storage nodes. If a device becomes inactive or disconnects from the network, the server detects the failure and redistributes the affected chunks to other available devices in order to maintain the required replication level.

When a user requests to retrieve a file, the system identifies the locations of all required chunks using metadata stored in the database. The server then collects the necessary chunks from the corresponding storage nodes and reassembles them in the correct order. The reconstructed encrypted file is sent back to the user device where it is decrypted and presented to the user.

To maintain strong security guarantees, the system follows a zero-trust security model. Storage nodes and the coordination server only handle encrypted data and do not have access to the original file contents. Encryption keys remain on the user device, ensuring that only the authorized user can decrypt and access stored files.

Through this architecture, the proposed system enables users to build a secure, decentralized, and cost-effective personal cloud infrastructure by leveraging the unused storage resources available on their own Android devices.

3.2 Architectural Design

The architectural design of the proposed system is illustrated using several diagrams that represent the interaction between users, devices, and the central coordination server. The system follows a decentralized storage model where user-owned mobile devices act as storage nodes, while a lightweight server manages metadata, coordination, and communication.

The architecture consists of three primary components: the Android application (User Mode and Storage Node Mode), the Central Coordination Server, and the distributed storage nodes. The Android application is responsible for file encryption, chunking, and user interaction, while the server handles authentication, metadata storage, device management, and chunk mapping. Storage nodes store encrypted chunks without access to

plaintext data, ensuring a zero-trust environment.

3.2.1 Use Case Diagram

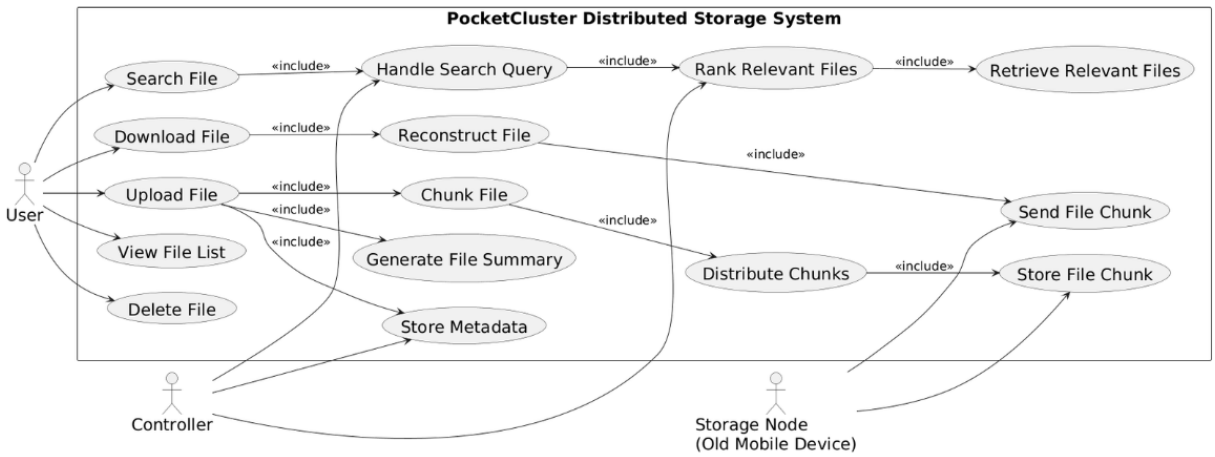


Figure 3.1: Use Case Diagram of the Proposed System

The use case diagram illustrates the interaction between the user and the distributed personal cloud system. It shows the major functionalities such as uploading files, retrieving files, registering devices, and enabling storage node participation.

Additionally, the diagram highlights the role of the central controller in coordinating operations such as chunk distribution, metadata management, and communication between devices. It also emphasizes the use of modular processes like chunking, reconstruction, and ranking, which are reused across multiple functionalities through ¡¡include¿¿ relationships. This structured interaction ensures scalability, efficient resource utilization, and secure handling of user data in a decentralized environment.

Furthermore, the separation of responsibilities between the user device, controller, and storage nodes improves system reliability and fault tolerance. The inclusion relationships also indicate that complex operations are broken down into smaller reusable components, simplifying system design and maintenance. This modular approach enables easier future enhancements and integration of additional features without affecting the overall system functionality.

3.2.2 Sequence Diagram

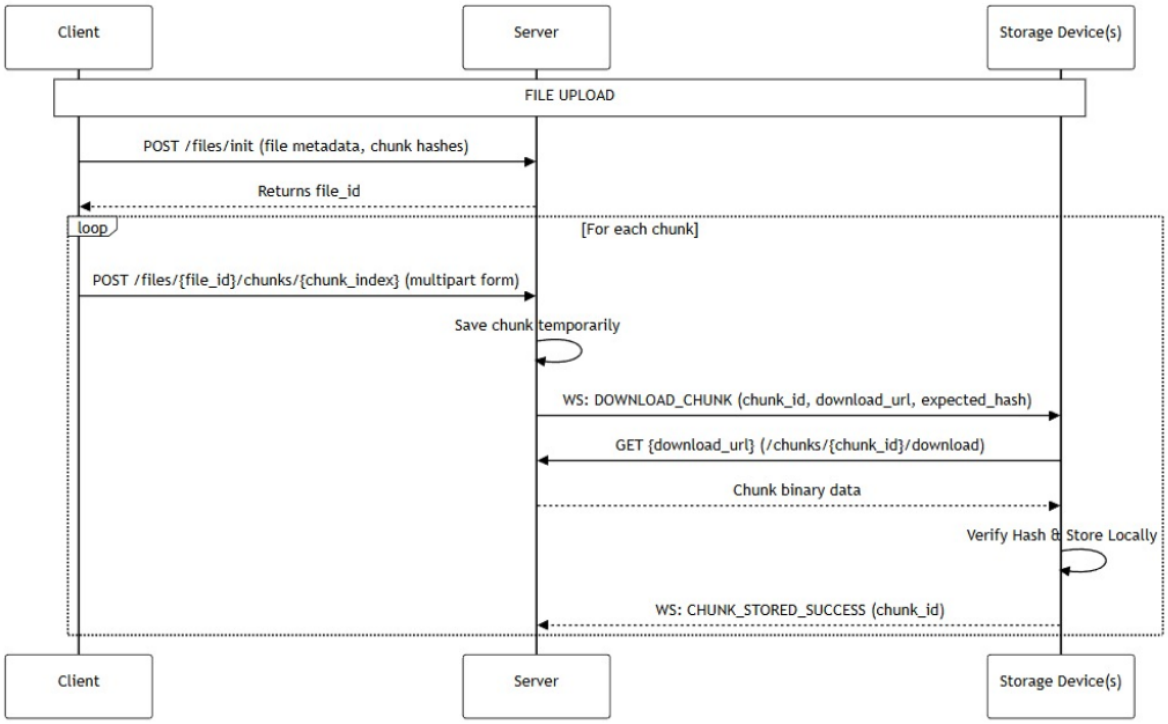


Figure 3.2: Sequence Diagram for File Upload Process

This sequence diagram illustrates the file upload process in the distributed storage system. Initially, the user initiates the upload operation, after which the file is processed and divided into smaller chunks on the user device. The chunk information is then sent to the central coordination server to prepare for storage.

The server selects suitable storage nodes based on availability and sends instructions to them to receive specific chunks. This selection ensures that only active and capable nodes participate in the storage process. Each chunk is then transferred from the user device to the respective storage nodes as directed by the server. The storage nodes store the received chunks and send confirmation messages back to the server.

Upon receiving acknowledgments, the server updates the status of each chunk in its metadata. This process helps in maintaining consistency and tracking the storage of all chunks efficiently. This ensures that all chunks are successfully stored and properly tracked within the system.

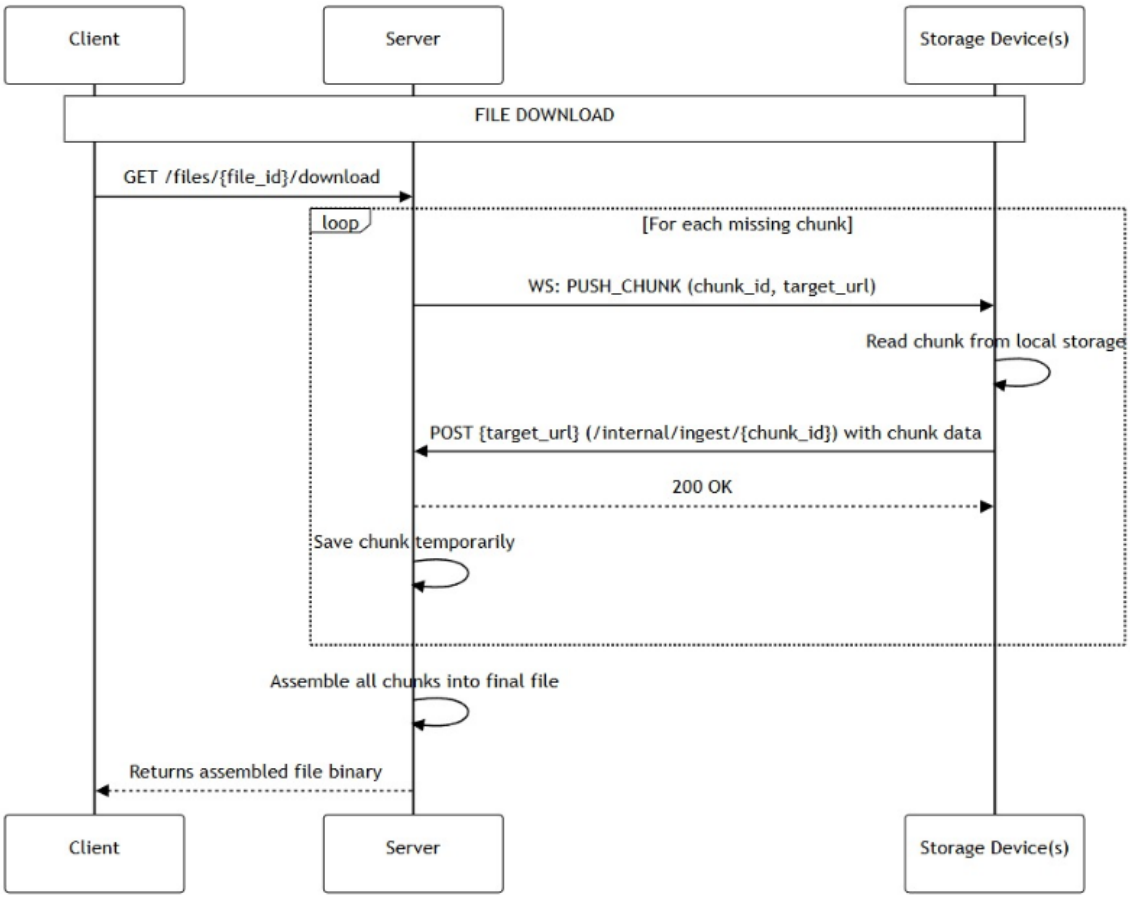


Figure 3.3: Sequence Diagram for File Retrieval Process

This sequence diagram illustrates the file retrieval process in the distributed storage system. Initially, the user initiates a request to download a file, which is received by the central coordination server. The server then identifies the required chunks and the storage nodes where they are available.

The server sends requests to the respective storage nodes to retrieve the required chunks. This selection ensures that only available nodes are contacted for data retrieval. Each storage node sends the corresponding chunk back to the server as requested. The chunks are transferred sequentially or in parallel depending on availability.

The server collects all the chunks and reconstructs the file in the correct order. This process ensures that the file is assembled properly before being delivered. Basic validation is performed to ensure all chunks are received successfully. Finally, the reconstructed file is sent back to the user, completing the download process.

3.2.3 System Architecture

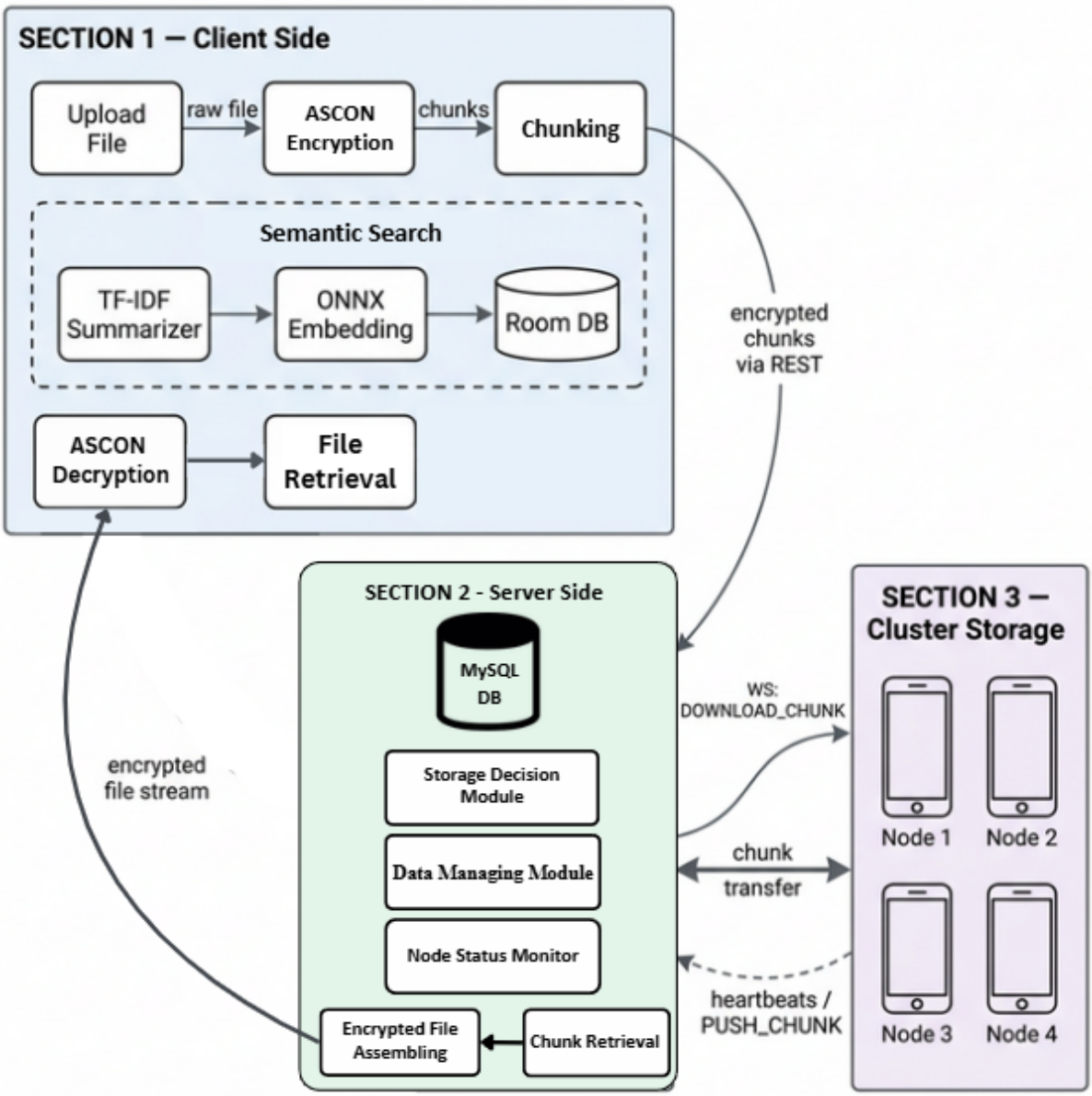


Figure 3.4: System Architecture of the Distributed Personal Cloud

The system architecture diagram presents the overall structure of the distributed personal cloud system, highlighting the interaction between Android devices operating in user mode and storage node mode, along with the central coordination server responsible for resource allocation and metadata management.

3.3 Dataset identified

The proposed system does not rely on a predefined external dataset for training or analysis. Instead, the system operates on files uploaded directly by users through the Android application. These files may include documents, images, text files, and other personal data stored by the user.

When a file is uploaded, it is processed locally on the user device. The file is encrypted and divided into fixed-size chunks before being transmitted to the coordination server for distribution across available storage nodes. Metadata such as file identifiers, chunk mappings, device information, and replication status are stored in the system database.

For semantic search functionality, textual content from uploaded files is processed to generate vector embeddings. A pre-trained embedding model, executed locally using Open Neural Network Exchange (ONNX) Runtime, is used to convert textual data into numerical vector representations. These embeddings are stored locally and used to perform similarity-based searches across stored files.

3.4 Proposed Methodology and Algorithms

The proposed distributed personal cloud system integrates multiple methodologies to ensure secure storage, efficient resource utilization, and reliable data retrieval across multiple Android devices. The system combines encryption techniques, distributed storage mechanisms, and dynamic resource allocation to create a decentralized storage environment.

Before storing files in the distributed network, the system ensures confidentiality by applying strong encryption mechanisms. The encrypted data is then divided into smaller chunks and distributed across multiple storage nodes. Replication mechanisms ensure fault tolerance and data availability even when some devices become unavailable.

The following algorithms describe the core processes implemented in the system, including encryption, chunking, resource allocation, and file retrieval.

Algorithm 1 ASCON AEAD-128 Encryption Process

Require: Plaintext file F, secret key K, nonce N

Ensure: Ciphertext C with authentication tag T

```
Read input file F from storage
Convert file data into binary representation
Divide the binary data into fixed-size message blocks m_1, m_2, ..., m_n
Initialize ASCON internal state S using key K and nonce N
Set initial parameters including rate and capacity
Apply initial permutation rounds on state S to randomize internal state
/* Associated Data Processing Phase */
If associated data A exists, split into blocks
for each associated data block a_i do
    XOR a_i with state S
    Apply permutation rounds to mix data into state
end for
Perform domain separation to distinguish data types
/* Encryption Phase */
for each message block m_i do
    XOR plaintext block m_i with current state S
    Extract ciphertext block c_i from modified state
    Store ciphertext block c_i
    Apply substitution layer (non-linear S-box transformation)
    Apply linear diffusion layer to ensure bit spreading
    Apply permutation rounds to update internal state
    Ensure state integrity before processing next block
end for
Concatenate all ciphertext blocks c_1, c_2, ..., c_n to form C
Mix secret key K into the state again for authentication
Apply multiple permutation rounds for final state diffusion
Extract authentication tag T from the final state
Append authentication tag T to ciphertext C
return (C, T)
```

Algorithm 2 ASCON AEAD-128 Decryption Algorithm

Require: Ciphertext C, authentication tag T, secret key K, nonce N

Ensure: Recovered plaintext file F

```
Read ciphertext C and authentication tag T
Convert ciphertext into binary and divide into blocks c_1, c_2, ..., c_n
Initialize ASCON internal state S using key K and nonce N
Apply initial permutation rounds to prepare state
/* Associated Data Processing */
for each associated data block a_i do
    XOR a_i with state S and apply permutation
end for
/* Decryption Phase */
for each ciphertext block c_i do
    Recover plaintext block m_i by XORing c_i with state S
    Store m_i and update state using c_i
    Apply substitution, diffusion, and permutation
    Verify intermediate state consistency before next iteration
end for
Concatenate plaintext blocks to form reconstructed data
Perform basic integrity check on reconstructed data
/* Finalization */
Mix key K into state and apply final permutation
Compute expected authentication tag T'
if T = T' then
    Accept data and reconstruct file F
else
    Reject ciphertext due to authentication failure
end if
return F
```

Algorithm 3 Encrypted File Chunking Algorithm

Require: Encrypted file E, predefined chunk size S

Ensure: Set of encrypted chunks C

```
Read encrypted file E from local storage
Validate file availability and accessibility
Convert file into binary stream representation
Determine total file size L
Initialize pointer p = 0
Initialize empty chunk list C
/* Chunking Process */
while p < L do
    Extract data segment d = E[p : p + S]
    Handle last chunk if remaining size < S
    Generate unique chunk identifier ID
    Compute hash value H(d) for integrity verification
    Encrypt metadata reference for secure identification
    Create chunk structure (ID, d, H(d))
    Store chunk in list C
    Log chunk creation status for tracking
    Update pointer p = p + S
end while
/* Metadata Preparation */
Generate file identifier FID
Record total number of chunks created
Create metadata containing FID, chunk IDs, and hash values
Attach timestamp for tracking chunk creation
/* Communication with Server */
Send metadata to coordination server via REST API
Await acknowledgment from server before proceeding
Retry metadata transmission in case of failure
return chunk set C
```

Algorithm 4 Dynamic Storage Node Allocation Algorithm

Require: Chunk metadata M, list of storage nodes N

Ensure: Allocation map for chunk replication

```
Receive chunk metadata from user device
Retrieve list of active storage nodes from registry
Initialize empty allocation table A
for each chunk c_i in metadata do
    Filter nodes with available storage capacity
    Sort eligible nodes based on free storage
    Select top k nodes for replication
    for each selected node n_j do
        Assign chunk c_i to node n_j
        Update allocation table A
    end for
end for
Store allocation information in metadata database
return allocation table A
```

Algorithm 5 Distributed File Retrieval Algorithm

Require: File request with file identifier F_{id}

Ensure: Reconstructed original file

```
Query metadata database for chunk mapping
Retrieve list of chunk identifiers C
Identify storage nodes containing replicas
for each chunk identifier c_i do
    Send retrieval request to corresponding node
    Receive encrypted chunk data
    Store received chunk in temporary buffer
end for
Sort chunks according to original sequence order
Concatenate all chunks to reconstruct encrypted file E
Apply ASCON decryption to obtain original file F
Return file F to user device
```

3.5 User Interface Design

The Android application provides a user-friendly interface that allows users to interact with the distributed personal cloud system efficiently. The interface is designed to provide easy access to core functionalities such as file upload, file browsing, participation as a storage node, and semantic search. The layout of the application is kept simple and intuitive so that users can easily navigate through the system and manage their stored files. The design also ensures minimal user effort by organizing features into clearly defined sections. Additionally, the interface follows standard Android design guidelines to maintain consistency and usability across devices.

Figure 3.5: Mode Selection Interface

The mode selection interface allows the user to choose between two operational modes of the application: *User Mode* and *Storage Node Mode*. In User Mode, the device acts as a client that uploads and retrieves files from the distributed cloud system. In Storage Node Mode, the device contributes its storage resources to the distributed cluster. This flexibility allows users to switch roles based on their requirements and available resources. The interface is designed to clearly distinguish between the two modes to avoid confusion.

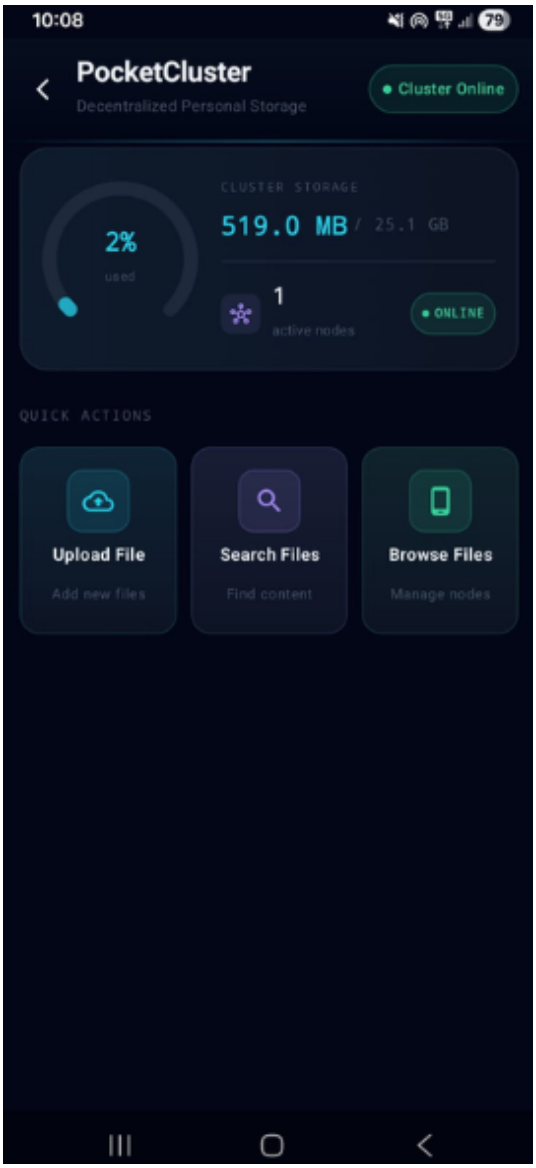


Figure 3.6: Storage Status Interface

The storage status interface displays information about the device storage usage. It shows how much storage space is currently used and how much storage remains available. This information helps the user monitor device storage when participating in the

distributed storage system. It also allows users to make informed decisions about contributing storage to the cluster. Real-time updates ensure that the displayed information remains accurate.

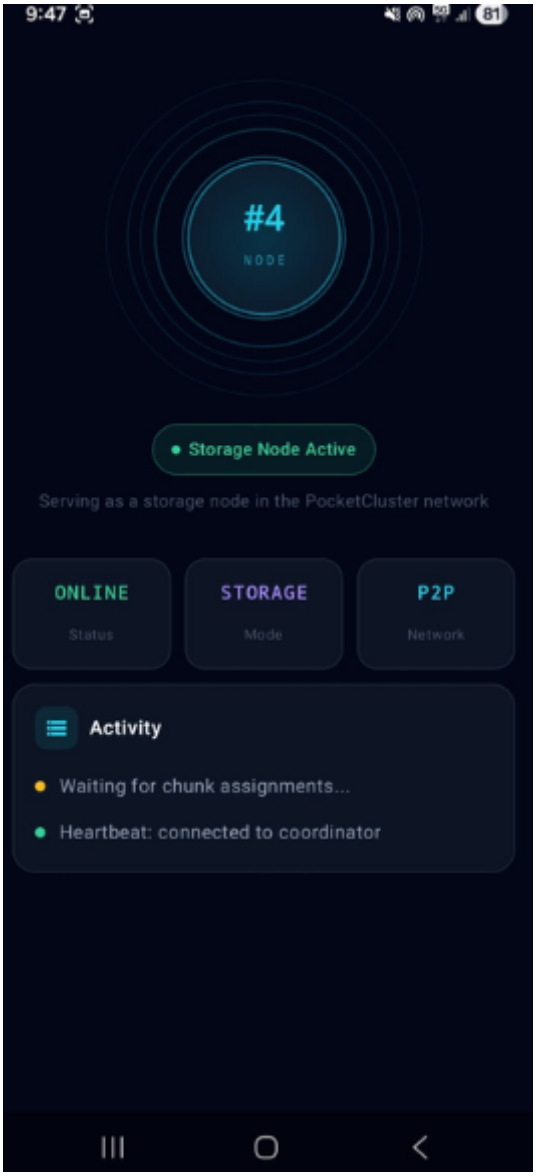


Figure 3.7: Storage Node Interface

The storage node interface is displayed when the device operates in storage node mode. This screen provides information such as the storage node identifier and the current status of the device within the distributed storage network. It also indicates whether the device is actively participating or temporarily offline. This helps users understand their device's role and contribution to the system. The interface ensures transparency in cluster participation.

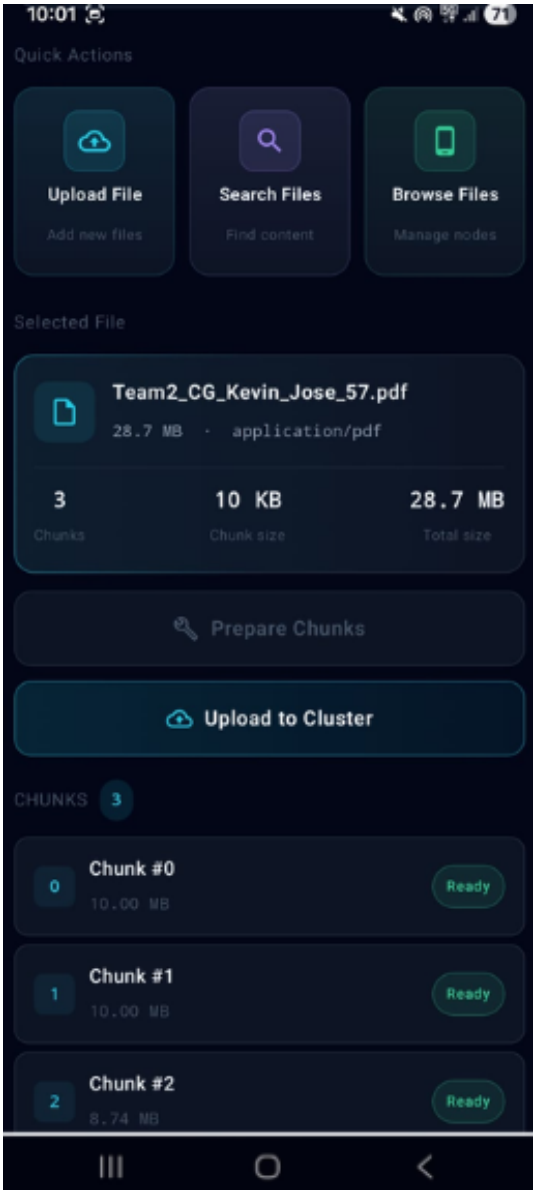


Figure 3.8: File Upload Interface

The file upload interface allows users to select files from the device storage and upload them to the distributed personal cloud. During the upload process, the selected file is encrypted locally and divided into smaller chunks before being distributed to multiple storage nodes. The interface provides feedback on upload progress to keep the user informed. This ensures a smooth and reliable upload experience. Error handling mechanisms are also incorporated to manage interruptions during upload. The system also ensures that only successfully processed chunks are sent for storage. This improves reliability and prevents partial uploads.

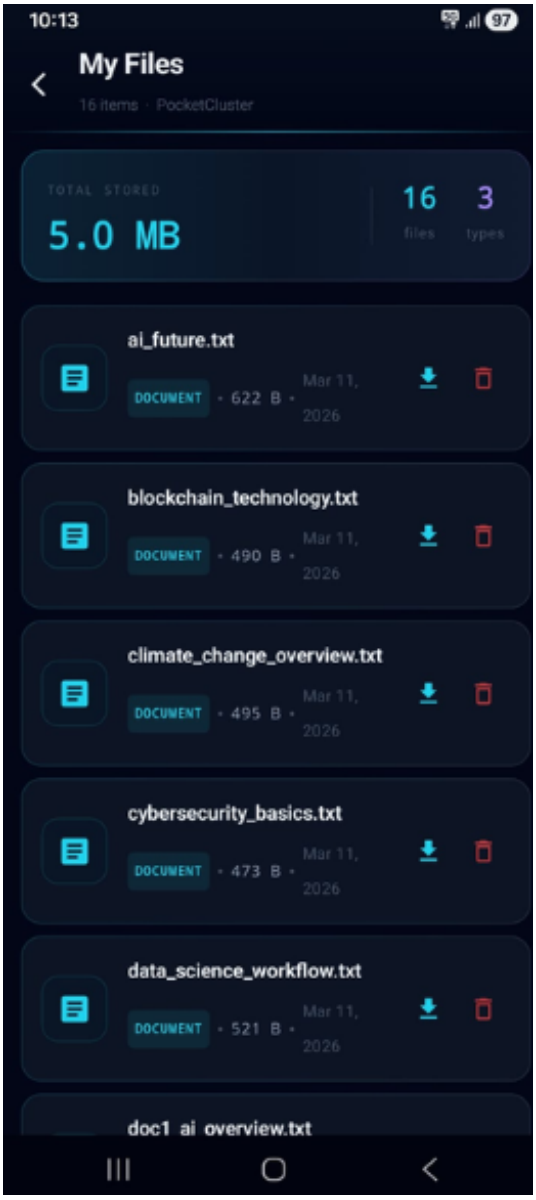


Figure 3.9: Browse Files Interface

The browse files interface displays the list of files that are stored in the distributed personal cloud. Users can view available files and select files for download or further operations. The interface organizes files in a structured manner for easy access. It also allows users to quickly identify file details such as name and type. This improves usability and enhances the overall user experience. Users can scroll through the file list efficiently without performance issues. The interface is optimized to handle large numbers of files smoothly. Additionally, basic file operations such as selection and navigation are designed to be responsive. This ensures a seamless browsing experience even on devices with limited resources.

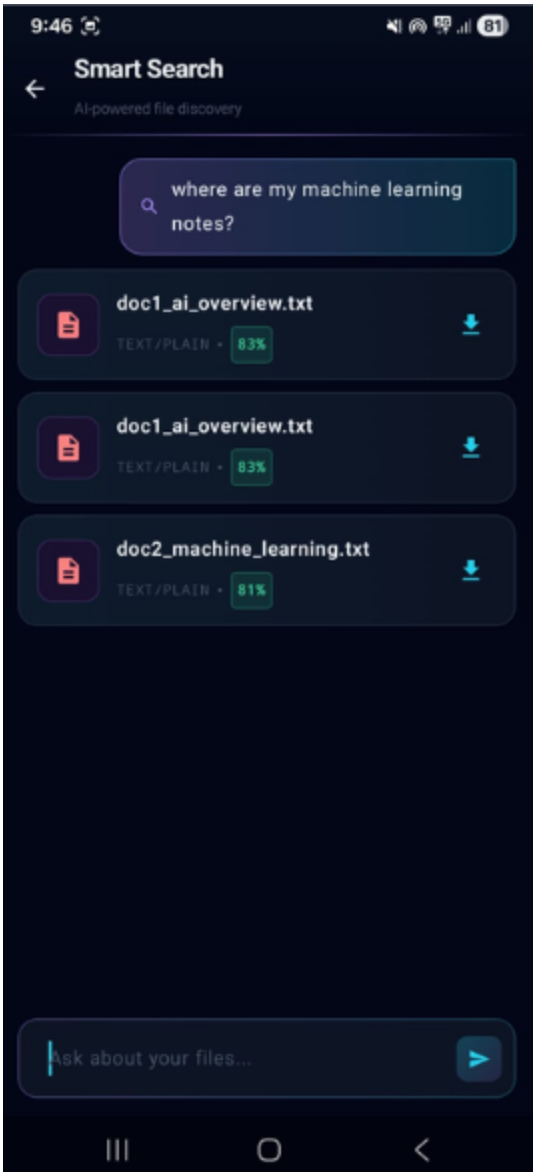


Figure 3.10: Semantic Search and File Download Interface

The semantic search interface allows users to search for stored files using contextual queries. The system analyzes the search query and retrieves files that are semantically related to the entered keywords. Users can then download the selected files from the distributed storage system. This feature improves search efficiency compared to traditional keyword-based search. It also enhances user experience by providing more relevant results. The search results are ranked based on similarity scores for better accuracy. This enables users to quickly locate the most relevant files. The system continuously refines search results based on stored embeddings. This further improves the relevance and quality of retrieved results over time.

3.6 Database Design

The distributed personal cloud system requires a structured database to manage metadata related to users, devices, files, and chunk replication across the storage network. The database does not store the actual file contents. Instead, it stores metadata that enables the system to locate distributed file chunks and reconstruct files when required.

A relational database management system is used to store and manage the metadata efficiently. The database schema supports core system operations such as user management, file uploads, chunk distribution, device monitoring, and file retrieval. Proper indexing and relational constraints are implemented to ensure data integrity and efficient query performance.

The database consists of five primary tables: *Users*, *Files*, *Devices*, *Chunks*, and *Chunk Replication*. The structure of each table is described below.

Users Table

The Users table stores information about registered users of the system including authentication details and account creation information.

Column Name	**Data Type**	**Description**
user_id	INT (PK)	Unique identifier for each user
email	VARCHAR(255)	User email address (unique)
password_hash	VARCHAR(255)	Encrypted password hash
created_at	TIMESTAMP	Account creation timestamp
last_login	TIMESTAMP	Last login time of the user

Table 3.1: Structure of the Users Table

Files Table

The Files table stores metadata related to files uploaded by users. Each file is associated with a specific user. It maintains essential information required for file identification, management, and retrieval within the distributed storage system. This table also helps in tracking file properties and coordinating chunk-level operations across storage nodes.

Column Name	Data Type	Description
file_id	INT (PK)	Unique identifier for each file
user_id	INT (FK)	References the user who uploaded the file
file_name	VARCHAR(255)	Name of the uploaded file
file_size	BIGINT	Size of the file in bytes
file_type	VARCHAR(100)	Type or format of the file
upload_timestamp	TIMESTAMP	Time when file was uploaded
num_chunks	INT	Number of chunks generated for the file

Table 3.2: Structure of the Files Table

Devices Table

The Devices table maintains information about Android devices participating in the distributed storage system. Devices can operate either in user mode or cluster (storage node) mode. It stores essential details required for device identification, status monitoring, and communication within the system. This table also plays a key role in tracking device availability and managing storage contributions in the distributed network.

Column Name	Data Type	Description
device_id	INT (PK)	Unique identifier for each device
user_id	INT (FK)	Associated user of the device
device_name	VARCHAR(255)	Name of the device
device_fingerprint	VARCHAR(255)	Unique identifier for the device
status	ENUM	Device status (ONLINE/OFFLINE)
mode	ENUM	Device mode (User or Cluster)
last_heartbeat	TIMESTAMP	Last communication timestamp
storage_capacity	BIGINT	Total storage capacity of the device
available_storage	BIGINT	Remaining available storage
created_at	TIMESTAMP	Device registration timestamp

Table 3.3: Structure of the Devices Table

Chunks Table

When a file is uploaded, it is encrypted and divided into smaller chunks. The Chunks table stores metadata associated with each chunk.

Column Name	Data Type	Description
chunk_id	INT (PK)	Unique identifier for each chunk
file_id	INT (FK)	Associated file identifier
chunk_index	INT	Sequence number of the chunk
chunk_hash	VARCHAR(255)	Hash value for chunk integrity
chunk_size	INT	Size of the chunk

Table 3.4: Structure of the Chunks Table

Chunk Replication Table

The Chunk Replication table manages the distribution and redundancy of chunks across multiple storage nodes.

Column Name	Data Type	Description
replication_id	INT (PK)	Unique replication record identifier
chunk_id	INT (FK)	Associated chunk identifier
device_id	INT (FK)	Device storing the chunk replica
replica_status	ENUM	Status of replica (Replicating, Active, Lost, Failed)

Table 3.5: Structure of the Chunk Replication Table

The relational design ensures that each file can be reconstructed by retrieving its chunks from multiple storage nodes. The replication mechanism increases reliability and fault tolerance by maintaining multiple copies of file chunks across different devices in the distributed system.

3.7 Description of Implementation Strategies

The distributed personal cloud system is implemented using a combination of mobile, server-side, and machine learning technologies. The implementation strategy focuses on

providing secure distributed storage, efficient file retrieval, and semantic search capabilities while utilizing the storage resources of participating Android devices. The system architecture follows a client-server model in which the Android application acts as the client while a Python-based backend server coordinates storage and retrieval operations.

3.7.1 Android Application

The front-end of the system is implemented as an Android application developed using Kotlin. The application provides an intuitive interface that allows users to upload files, browse stored files, perform semantic searches, and download files. The application also allows a device to operate either in *User Mode* or *Cluster Mode*. In User Mode, the device acts as a client interacting with the distributed storage system. In Cluster Mode, the device contributes its storage resources to store file chunks from other users.

The Android application is responsible for performing several local operations including file encryption, file chunking, embedding generation for semantic search, and communication with the backend server.

3.7.2 Backend Server

The backend server is implemented using Python and is responsible for coordinating the distributed storage system. The server manages user authentication, metadata storage, device registration, and chunk distribution. When a file is uploaded, the server determines suitable storage nodes based on their available storage capacity and device status.

The backend also maintains the metadata database, which stores information about users, files, chunks, and chunk replication across storage devices. During file retrieval, the server identifies the devices that contain the required chunks and coordinates the reconstruction of the original file.

3.7.3 REST API Communication

Communication between the Android application and the backend server is performed using RESTful APIs. These APIs allow the client application to send requests such as user login, file upload, file metadata retrieval, and semantic search queries. The server processes these requests and responds with the necessary metadata or storage instructions.

REST APIs provide a standardized communication mechanism and ensure interoperability between the mobile client and the backend system.

3.7.4 File Encryption Using ASCON

To ensure data security and privacy, files are encrypted before being distributed across storage nodes. The system uses the ASCON authenticated encryption algorithm, which is a lightweight cryptographic algorithm designed for secure communication in resource-constrained environments.

Before a file is divided into chunks, it is encrypted locally on the user's device using ASCON encryption. This ensures that storage nodes cannot access the original file content. During file retrieval, the encrypted chunks are reconstructed and then decrypted on the user's device using the corresponding decryption key.

3.7.5 File Chunking and Distribution

After encryption, the file is divided into smaller segments called chunks. Chunking allows the system to distribute file data across multiple devices instead of storing the entire file in a single location. Each chunk is assigned a unique index and hash value to maintain ordering and ensure data integrity.

The backend server assigns storage nodes for each chunk based on available storage capacity and device availability. Multiple replicas of chunks are stored across different devices to ensure reliability and fault tolerance.

3.7.6 Semantic Search Using Embedding Models

To enable intelligent file retrieval, the system incorporates a semantic search mechanism. Instead of relying solely on exact keyword matching, semantic search allows users to retrieve files based on contextual meaning.

When a file is uploaded, its metadata such as file name and description is processed using a lightweight embedding model exported in ONNX format. The embedding model converts textual information into numerical vector representations. These vectors capture the semantic meaning of the text.

When a user performs a search query, the query text is also converted into an embedding vector. The system then compares this query vector with stored file embeddings

using similarity measures such as cosine similarity. Files with the highest similarity scores are returned as relevant results.

3.7.7 Distributed Storage Coordination

The system continuously monitors the availability and storage capacity of participating devices using periodic heartbeat signals. Devices operating in Cluster Mode report their status to the server, allowing the system to maintain an updated view of the distributed storage network.

If a device goes offline, the system can identify missing chunk replicas and redistribute them to other available devices to maintain redundancy and data availability.

Through the combination of secure encryption, distributed storage management, and semantic search capabilities, the implementation strategy enables the system to function as a decentralized personal cloud storage platform.

3.8 Module Division

The project is structured into several interconnected modules, each handling a specific aspect of the system's functionality. These modules collectively enable secure decentralized storage, replication, and semantic search on user devices while maintaining coordination via the central server.

3.8.1 Android App Module

The Android App Module forms the client-side of the system and is implemented in Kotlin. It operates in two modes: User Node and Storage Node. In User Node mode, the application allows users to upload files, perform semantic search, and retrieve stored data. In Storage Node mode, the device participates in the cluster, storing encrypted file chunks and serving them to other users. This module also handles client-side encryption, chunking, local embedding generation, and integration with RoomDB for storing metadata and embeddings. Foreground services ensure that background operations such as heartbeats, chunk serving, and WS communication continue even when the app is minimized.

3.8.2 Backend Module

The Backend Module is implemented using FastAPI and serves as the central coordination server. It exposes both RESTful APIs and WebSocket endpoints for communication with Android clients. The server manages authentication, device registration, metadata storage, and routing of file chunks. It coordinates chunk replication, monitors device availability through heartbeat signals, and oversees the repair of lost or corrupted chunks. The module ensures that all operations maintain end-to-end encryption and zero-trust principles.

3.8.3 WebSocket Module

The WebSocket Module handles persistent bidirectional communication between the server and cluster devices. It maintains connections for heartbeat monitoring, chunk assignment, replication commands, and control messages. Each device authenticates upon connection, and the server maps the device ID to its corresponding WebSocket session.

A key challenge in coordinating mobile devices is that most smartphones operate behind Network Address Translation (NAT) in mobile or home networks, which prevents direct inbound connections from the server. To address this limitation, devices initiate an outbound WebSocket connection to the server, allowing a persistent communication channel to be established. Through this channel, the server can send real-time instructions and receive status updates without requiring direct device addressing.

This module is critical for real-time coordination of the cluster and reliable chunk distribution.

3.8.4 Services Module

The Services Module encapsulates the core business logic of the system. It includes components for chunk distribution, replication planning, repair management, offline detection, and fault tolerance. This layer ensures that data remains available and consistent across the cluster, even in the presence of node failures or network interruptions. It abstracts the complexities of storage management from both the client and server APIs.

3.8.5 Data Models Module

The Data Models Module defines the database schema and Object Relational Mapping (ORM) mappings using SQLAlchemy. Each table, such as users, devices, files, and chunk replication, is represented as a Python class. This module provides a structured interface for database operations, handling object-to-SQL translation, transaction management, and session pooling. It ensures data integrity and simplifies interaction with the relational database.

3.8.6 Utilities Module

The Utilities Module includes helper components for encryption and decryption (ASCON), chunk hashing, chunking and reassembly, device information retrieval, local storage management, and HTTP/REST client utilities. These utilities are used both by the Android App and backend services to ensure secure, reliable, and efficient operations across the system.

3.8.7 Semantic Search Module

This module provides local text processing, embedding generation, and semantic search capabilities on the user device. It implements a Term Frequency–Inverse Document Frequency (TF-IDF) summarizer for document pre-processing and ONNX-based sentence embeddings using the all-MiniLM-L6-v2 model. The module generates semantic embeddings locally on the device and performs similarity scoring for search queries. The results and embeddings are stored in RoomDB to enable fast and persistent search operations without uploading sensitive content to the server, thereby preserving user privacy.

3.8.8 Core Infrastructure Module

The Core Infrastructure Module contains foundational components used across the system. It includes logging, configuration management, error handling, and shared constants. This module also houses the core backend utilities that facilitate communication between services, handle session management, and provide the structural backbone for both the Android and server components.

3.8.9 Foreground Service Module

Dedicated to Android Storage Node operations, the Foreground Service Module ensures that cluster devices remain operational and responsive at all times. It integrates heartbeat transmission, WebSocket communication, chunk serving, and repair management into a single service, keeping the cluster functional even when the app is not in the foreground or has been removed from recent tasks.

3.8.10 File Management Module

This module focuses on file handling, including uploading, downloading, chunking, reassembly, and metadata tracking. It ensures that encrypted chunks are correctly distributed, validated using hashes, and reassembled on retrieval. The module also handles interrupted uploads, retry mechanisms, and guarantees that only active replicas are used during file reconstruction.

3.9 Work Schedule-Gantt Chart

The work schedule for the project is illustrated using a Gantt chart. This chart summarizes the timeline of tasks, milestones, and development phases, providing a clear overview of the project planning, execution, and progress tracking.The project was systematically divided into multiple phases to ensure structured development and efficient time management. The initial phase involved problem identification and requirement analysis, where the limitations of centralized cloud storage systems were studied and the need for a decentralized, user-controlled storage system was established.Following this, the system design phase focused on defining the architecture of the proposed solution, including the Central Coordination Server, Android application with dual modes (User Mode and Storage Node Mode), and the communication mechanisms using REST APIs and WebSockets. Security design aspects such as client-side encryption, zero-trust storage, and metadata-based access control were also planned during this phase.The development phase was carried out in parallel modules. The backend server was implemented using FastAPI, handling device registration, metadata management, heartbeat monitoring, chunk replication, and file retrieval. Simultaneously, the Android application was developed using Kotlin, implementing features such as device registration, file chunking, encryption, foreground services

for storage nodes, and WebSocket-based communication.

The integration phase involved connecting the Android application with the backend server, ensuring seamless communication between user devices and storage nodes. Special attention was given to real-time coordination, handling network constraints such as NAT, and ensuring reliable chunk transfer and replication.Testing and validation were conducted throughout the development process. This included testing device availability using heartbeat mechanisms, verifying chunk integrity using hashing, ensuring correct replication, and validating secure file upload and retrieval processes. Edge cases such as device disconnection, failed replication, and corrupted chunks were also handled.Finally, the system was refined and optimized based on testing results, and additional features such as semantic search using ONNX models and improved user interface components were integrated.

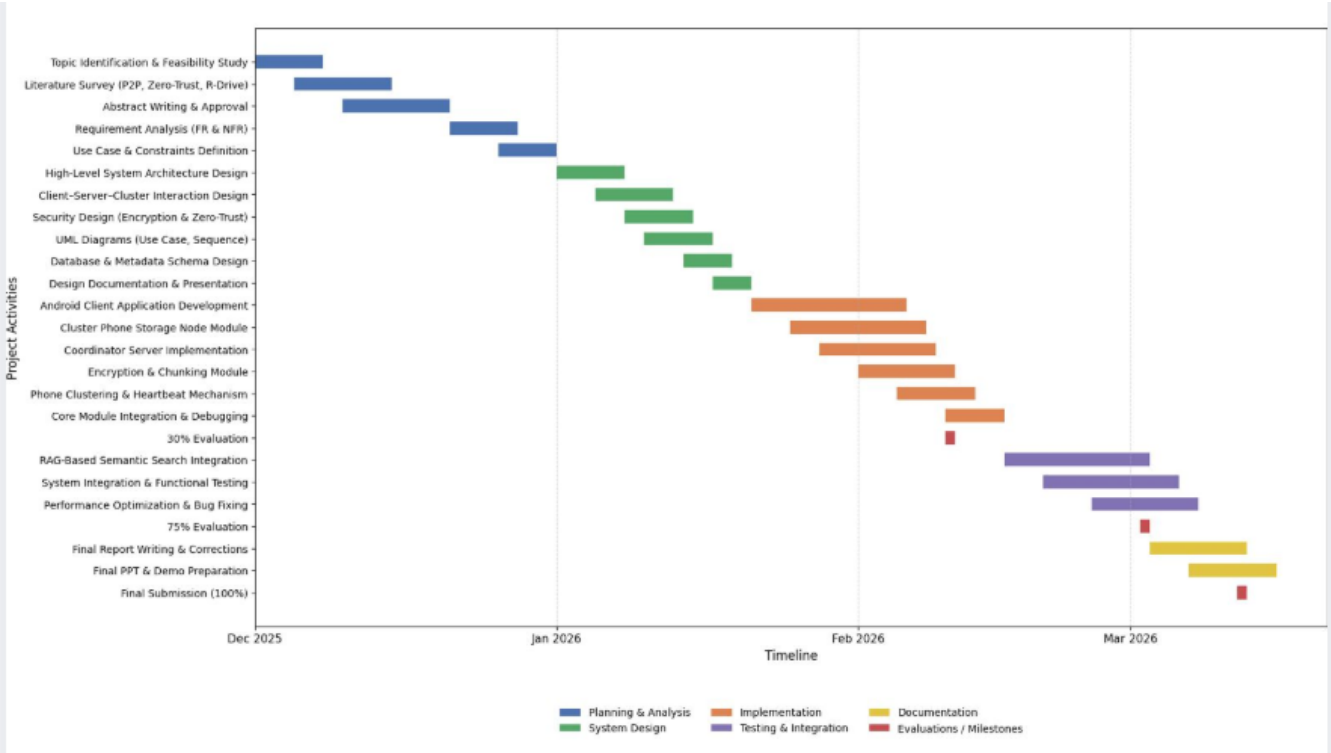


Figure 3.11: Project Work Schedule Gantt Chart

As shown in Figure 3.11, the project tasks are divided into phases including system design, module development, testing, and integration. Each phase is allocated specific timelines and highlighted to ensure smooth progress and timely completion of the project.

Chapter 4

Results and Discussions

4.1 Overview of Results

The experimental evaluation of the decentralized mobile cloud system demonstrates its ability to securely store and manage files across user-owned Android devices. The system was tested for upload latency, chunk distribution, and semantic search performance. Key observations include the impact of chunk size on upload latency, showing a trade-off between transmission overhead and storage efficiency, and the breakdown of semantic search latency, highlighting the computational cost of generating embeddings locally using the all-MiniLM-L6-v2 ONNX model. The administrative monitoring interface accurately reflects device statuses (ONLINE, OFFLINE) and chunk health (ACTIVE, REPLICATING, FAILED, LOST), confirming the reliability of the system. Overall, the results validate that the proposed approach provides secure, user-controlled storage with predictable performance while preserving data privacy and system integrity.

4.2 Testing

The proposed decentralized mobile cloud storage system was tested using a combination of Android emulators and physical Android devices acting as storage nodes. The backend coordination server was deployed locally and accessed through a shared network connection. The testing phase focused on verifying the functionality of the system components including device registration, cluster node monitoring, file upload, chunk replication, file retrieval, and semantic search.

Device registration was tested by allowing Android devices to send device metadata such as device fingerprint, device name, total storage capacity, and available storage to the coordination server through a REST endpoint. The server validated the uniqueness of the device fingerprint and assigned a unique device identifier which was stored locally

on the device for future communication.

Storage node functionality was verified by enabling the storage mode in the Android application. When activated, the device established a persistent WebSocket connection with the server and began transmitting periodic heartbeat signals to indicate its availability. The server monitored these heartbeats to determine whether a device was online or offline.

File upload testing involved selecting files through the Android application interface. The files were divided into fixed-size chunks and encrypted using ASCON AEAD-128 encryption on the client device. The encrypted chunks were transmitted to the server where they were temporarily stored and then distributed to active storage nodes for replication. Hash validation was used to verify the integrity of each chunk.

File retrieval testing was conducted by requesting stored files from the user interface. The server collected the required chunks from active storage replicas, reconstructed the file, and delivered it back to the user device for decryption.

An administrative monitoring interface was also implemented to observe the real-time status of cluster devices and replication health. The interface displays device connectivity states such as *ONLINE* and *OFFLINE*, along with replication states including *ACTIVE*, *REPLICATING*, *FAILED*, and *LOST*. This interface was used during testing to confirm that the system correctly detects device availability and manages chunk replication.

Figure 4.1: Administrative interface displaying device status and replication health in the storage cluster.

4.3 Quantitative Results

The quantitative evaluation of the system focused on measuring the latency characteristics of key operations in the decentralized storage architecture. Two primary performance metrics were analyzed: the effect of chunk size on upload latency and the latency breakdown of the semantic search functionality.

The first experiment evaluated how chunk size affects the time required to upload file segments to the system. Files were divided into different chunk sizes and the time required for processing and transmitting each chunk was measured. The results demonstrate that larger chunk sizes increase the latency due to the additional encryption and transmission overhead associated with each chunk. However, excessively small chunk sizes also introduce overhead because a larger number of chunks must be processed and managed by the server. Therefore, selecting an optimal chunk size is important for balancing processing overhead and transmission efficiency.

The second experiment measured the latency involved in the semantic search pipeline implemented on the user device. The semantic search operation consists of multiple stages including text preprocessing, summarization, embedding generation, and similarity comparison. The latency breakdown graph illustrates the contribution of each stage to the overall search response time.

The experimental results indicate that embedding generation and similarity computation contribute significantly to the total latency, while preprocessing and summarization introduce comparatively smaller overhead. Despite this, the overall search process remains efficient as all computations are performed locally on the user device without transmitting file contents to external servers, thereby preserving user privacy.

4.4 Graphical Analysis

Graphical analysis was conducted to evaluate the performance characteristics of the proposed system. Two key experiments were performed to analyze system latency under different operational conditions. These experiments include the analysis of chunk upload latency with respect to chunk size and the latency breakdown of the semantic search pipeline implemented on the user device.

The first experiment analyzed the relationship between chunk size and upload latency.

Files were divided into chunks of varying sizes and the time required to encrypt and upload each chunk was measured. The results demonstrate that larger chunk sizes generally lead to increased latency due to higher encryption and transmission overhead for each chunk. However, very small chunk sizes can also introduce additional overhead because a larger number of chunks must be managed and replicated by the server. Therefore, selecting an optimal chunk size is important to balance encryption cost, network transmission time, and chunk management overhead.

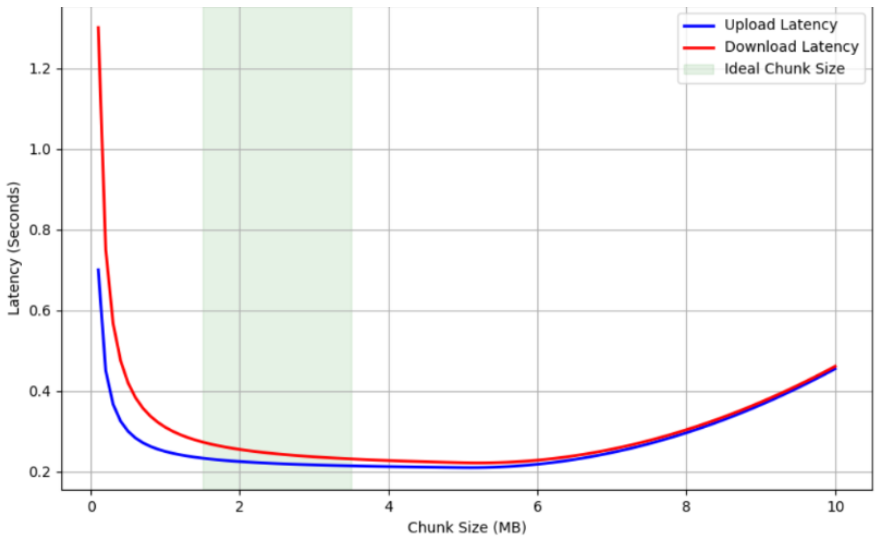


Figure 4.2: Relationship between chunk size and upload latency in the distributed storage system.

The second experiment evaluated the latency involved in performing semantic search on the user device. The semantic search pipeline consists of multiple processing stages including text preprocessing, summarization, embedding generation using the ONNX-based sentence transformer model, and similarity comparison with stored embeddings.

The latency breakdown graph illustrates the contribution of each stage to the overall search response time. The results indicate that embedding generation contributes the largest portion of the latency due to the computational requirements of running the neural model on the mobile device. Despite this overhead, the overall search latency remains acceptable for interactive use while preserving user privacy, since both the file content and embeddings remain entirely on the user device.

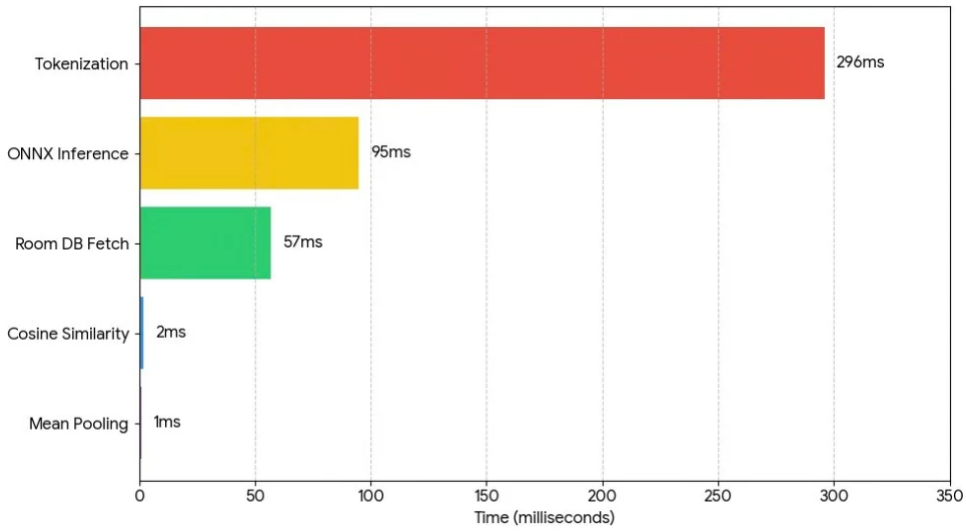


Figure 4.3: Latency breakdown of different stages involved in the semantic search pipeline.

4.5 Discussion

The experimental results demonstrate that the proposed decentralized mobile cloud storage system can securely manage distributed storage across user-owned mobile devices while maintaining acceptable performance. The chunk-based storage mechanism enables reliable data distribution across multiple nodes, while client-side encryption ensures data confidentiality. Replication further improves reliability by maintaining multiple copies of each chunk across different devices.

The administrative monitoring interface confirms that the system can effectively track device availability and replication health through statuses such as ACTIVE, REPLICATING, FAILED, and LOST.

Latency analysis shows that chunk size significantly influences system performance. Larger chunks reduce chunk management overhead but increase encryption and transmission latency, indicating the need for an optimal chunk size selection.

The semantic search latency breakdown highlights the computational cost of generating embeddings on mobile devices. Although this introduces additional processing time compared to server-side approaches, it preserves privacy by ensuring that file contents and embeddings remain on the user's device.

Overall, the system successfully achieves its goal of providing a secure, user-controlled personal cloud storage solution with reliable performance and strong privacy guarantees.

Chapter 5

Conclusion

5.1 Conclusion

This project presented the design and implementation of a decentralized personal mobile cloud storage system that utilizes user-owned mobile devices as distributed storage nodes. The system aims to provide a secure and privacy-preserving alternative to traditional centralized cloud storage services by eliminating the need for third-party storage providers. A central coordination server was implemented to manage metadata, device status, and chunk mapping, while actual file data is stored across user-controlled devices within a storage cluster.

The proposed system incorporates client-side encryption using ASCON AEAD-128, ensuring that files are encrypted before leaving the user device. As a result, storage nodes and the coordination server operate under a zero-trust model where they store only encrypted data without access to plaintext file contents. Files are divided into fixed-size chunks and replicated across multiple storage nodes to improve reliability and availability. A heartbeat-based monitoring mechanism was implemented to track node availability and trigger automated repair processes when replicas are lost.

Experimental evaluation demonstrated that the system successfully manages distributed storage across multiple devices while maintaining data integrity and availability. The latency analysis of chunk uploads highlights the trade-offs involved in selecting chunk sizes, while the semantic search latency breakdown demonstrates the feasibility of performing intelligent search operations locally on user devices. Overall, the proposed system provides a secure, user-controlled storage solution that leverages existing mobile devices to create a decentralized personal cloud environment.

5.2 Future Scope

Although the proposed system demonstrates the feasibility of decentralized mobile cloud storage, several improvements can be explored in future work. One possible extension is the development of an automated deployment system that simplifies the setup of the coordination server on cloud platforms. Additionally, advanced replication strategies could be implemented to dynamically adjust the number of replicas based on device availability and network conditions.

Future work may also focus on improving system scalability by supporting larger storage clusters and optimizing chunk distribution algorithms. Enhancing the semantic search component with more advanced embedding models could further improve search accuracy and performance. Finally, improvements to the user interface and device management features can provide a more seamless experience for users managing their personal storage clusters.

References

[1] M. Sagor, R. Stoleru, S. Bhunia, M. Chao, A. Haroon, A. Altaweel, M. Maurice, and R. Blalock, "R-Drive: Resilient data storage and sharing for mobile edge computing systems," arXiv preprint arXiv:2204.10823, Apr. 2022. doi: 10.48550/arXiv.2204.10823.

[2] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "Ascon v1.2: Lightweight Authenticated Encryption and Hashing," *Journal of Cryptology*, 2021.

[3] K. Sonbol, Ö. Özkasap, I. Al-Oqily, and M. Aloqaily, "EdgeKV: Decentralized, scalable, and consistent storage for the edge," arXiv preprint arXiv:2006.15594, 2020.

[4] S. K. Monga, S. K. R., and Y. Simmhan, "ElfStore: A Resilient Data Storage Service for Federated Edge and Fog Resources," arXiv preprint arXiv:1905.08932, 2019.

[5] B. Warner, " Tahoe-LAFS: Secure, decentralized, fault-tolerant distributed storage system," Proceedings of the ACM Storage Security Symposium, 2008.

[6] M. Satyanarayanan, P. Bahl, R. Caceres, and N. Davies, "The Case for VM-Based Cloudlets in Mobile Computing," *IEEE Pervasive Computing*, vol. 8, no. 4, pp. 14–23, 2009.

[7] H. Li, G. Shou, Y. Hu, and Z. Guo, "Mobile Edge Computing: Progress and Challenges," IEEE International Conference on Mobile Cloud Computing, 2016.

[8] Y. Pan, Y. Li, C. Gao, L. Fang, and P. Chen, "Flexible and Efficient Blockchain-Based Cloud Storage," IEEE International Conference on Cloud Computing, 2021.

[9] X. He and W. Dou, "Offloading Deadline-aware Task in Edge Computing," IEEE International Conference on Cloud Computing, 2020.

[10] R. Jain and S. Tata, "Cloud to Edge: Distributed Deployment of Process-Aware IoT Applications," IEEE International Conference on Edge Computing, 2017.

Appendix A: Presentation

FINAL PRESENTATION

Dynamic Resource Aggregation and Allocation Across Multiple Android Devices for a Distributed Personal Cloud

Joel Sony
Joseph Thankachan
Kevin Jose
David George

Ms. Seema Safar

POCKETCLUSTER

CONTENTS

1. Introduction
2. Problem Definition
3. Objectives
4. Scope and Relevance
5. System Design
6. Work Division – Gantt Chart
7. Software/Hardware Requirements
8. Results
9. Conclusion
10. Future Enhancements
11. References

INTRODUCTION

- Millions of functional smartphones are discarded annually, creating a massive e-waste stream. Traditional recycling recovers only a fraction of these high-value components, leading to significant environmental degradation.
- The Solution Instead of discarding old devices, this project repurposes them into a decentralized personal cloud. By clustering idle phones, we transform underutilized hardware into a functional distributed storage network.

PROBLEM DEFINITION

To design and implement a secure distributed storage system that repurposes discarded smartphones as decentralized storage nodes, providing a viable solution to reduce electronic waste through functional device reuse.

OBJECTIVE

Develop a dual-mode mobile application that leverages a zero-trust hybrid-P2P architecture and client-side encryption to provide a cost-effective, private storage cluster coordinated by a secure central server.

SCOPE AND RELEVANCE

Scope

- Personal decentralized cloud storage using old Android devices
- Central coordinator server for metadata and device management

Key Features

- Zero-trust architecture (server cannot read user data)
- Client-side encryption + file chunking
- Chunk replication across devices

Why It Matters

- Reuses old smartphones → reduces e-waste
- Privacy-focused storage model

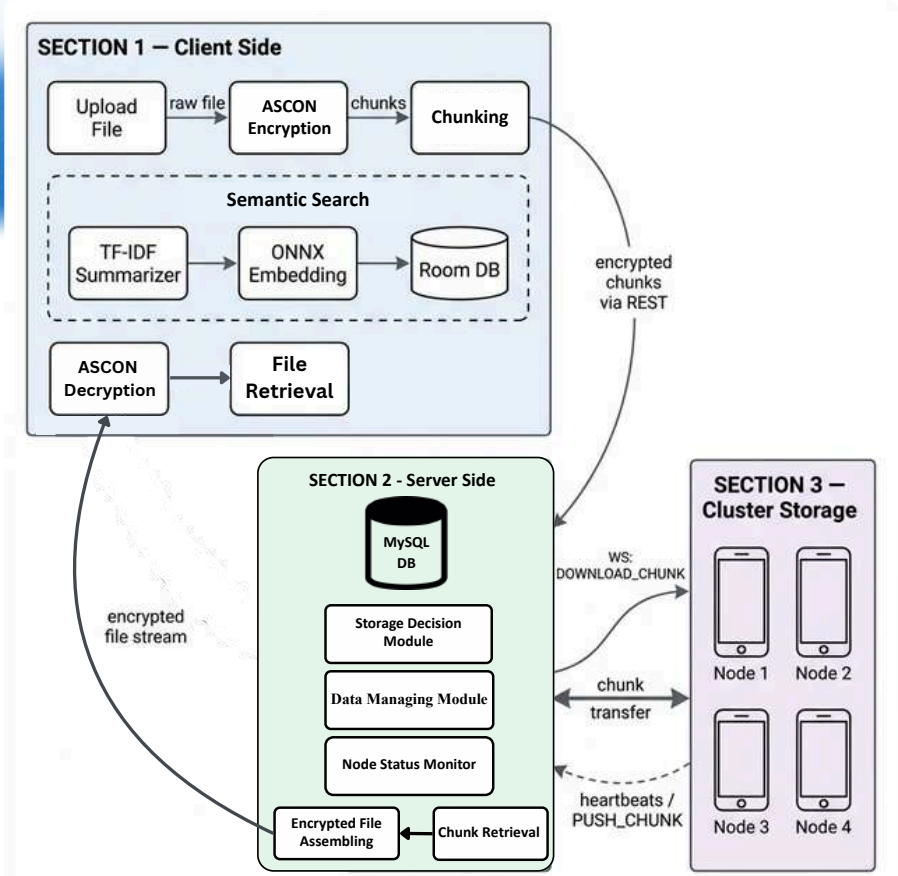
Applications

- Personal cloud storage using owned devices
- Research platform for decentralized storage systems

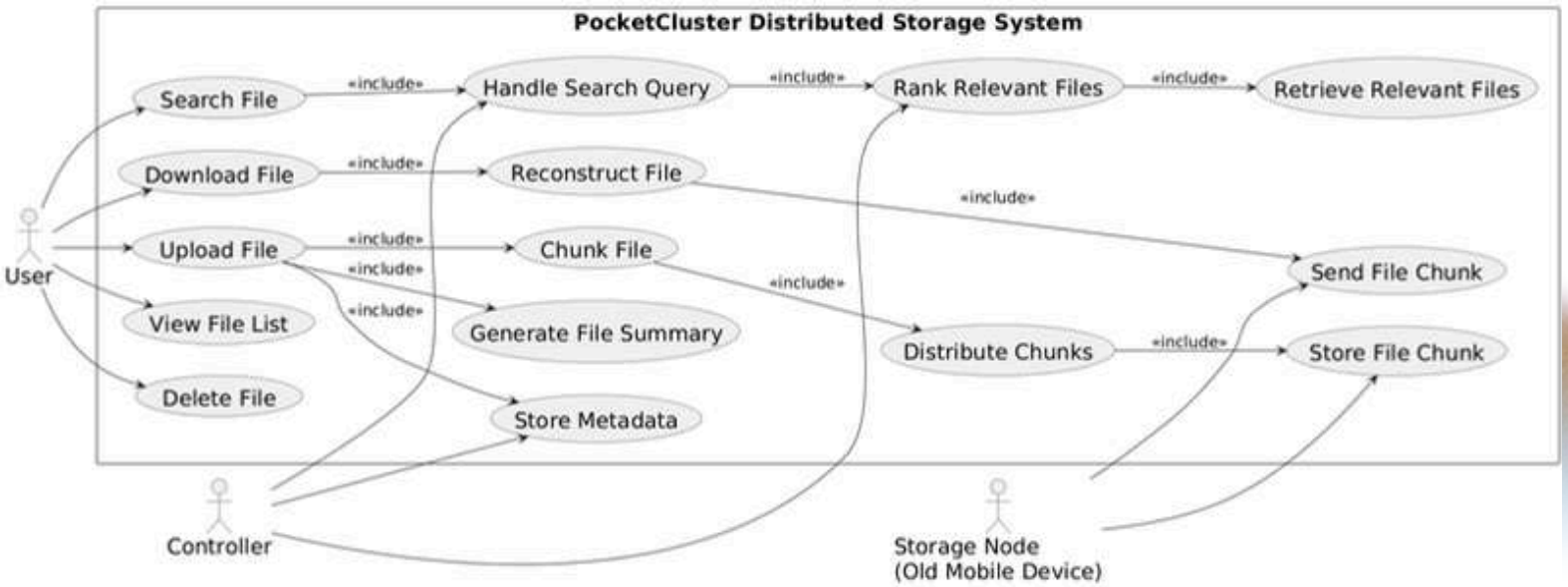
ASSUMPTIONS

- A person is supposed to handle the initial setup of the server.
- All the devices should be connected to the same network.
- The storage nodes should be running at all times, or else will be considered disconnected.

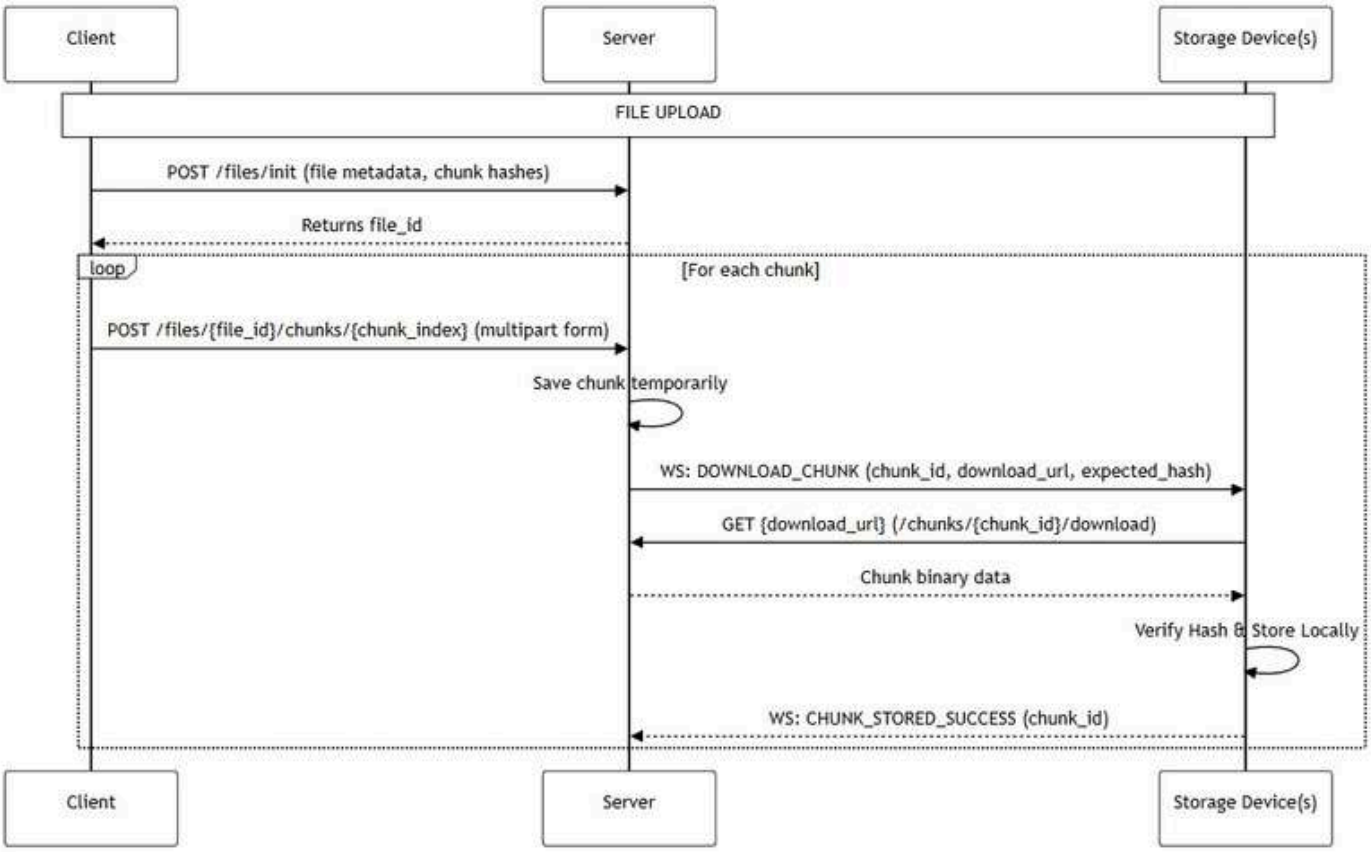
SYSTEM ARCHITECTURE

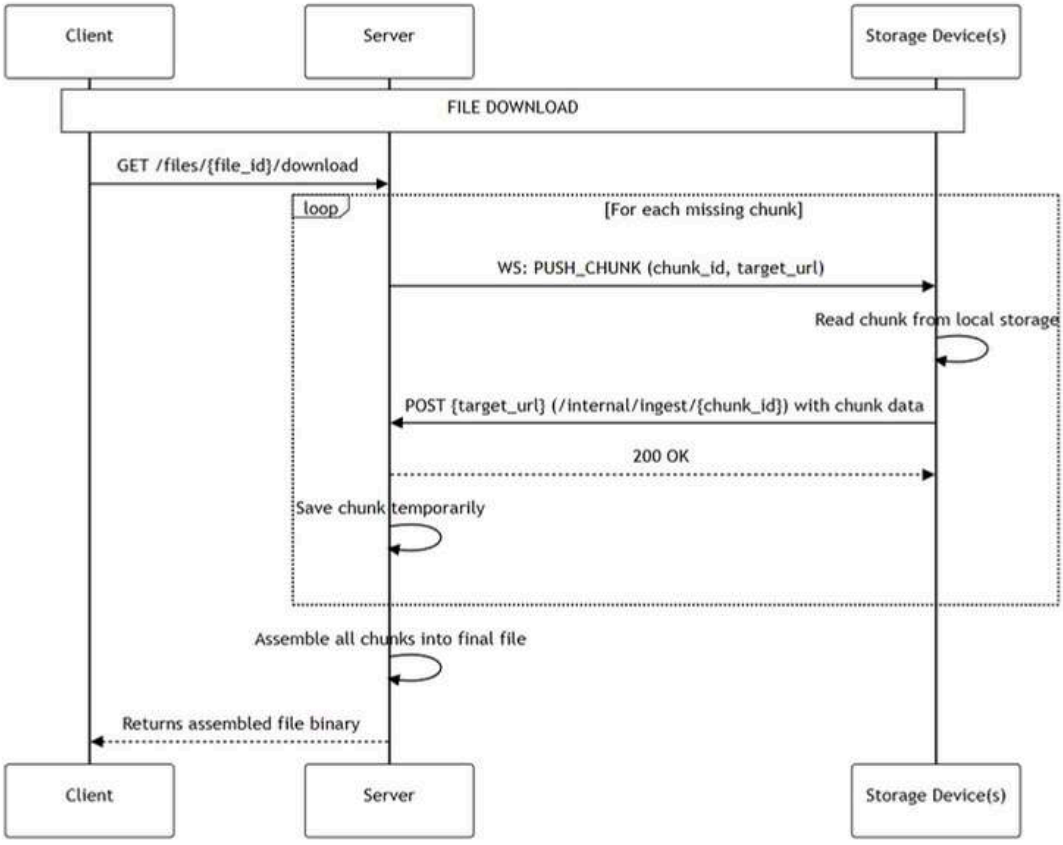


Use Case Diagram



SEQUENCE DIAGRAM


Client
Server
Storage Device(s)
FILE UPLOAD
POST /files/init (file metadata, chunk hashes)
Returns file_id
loop
[For each chunk]
POST /files/{file_id}/chunks/{chunk_index} (multipart form)
Save chunk temporarily
WS: DOWNLOAD_CHUNK (chunk_id, download_url, expected_hash)
GET {download_url} (/chunks/{chunk_id}/download)
Chunk binary data
Verify Hash & Store Locally
WS: CHUNK_STORED_SUCCESS (chunk_id)
Client
Server
Storage Device(s)


Client
Server
Storage Device(s)
FILE DOWNLOAD
GET /files/{file_id}/download
loop
[For each missing chunk]
WS: PUSH_CHUNK (chunk_id, target_url)
Read chunk from local storage
POST {target_url} (/internal/ingest/{chunk_id}) with chunk data
200 OK
Save chunk temporarily
Assemble all chunks into final file
Returns assembled file binary
Client
Server
Storage Device(s)

MODULES

The system consists of two major components: a FastAPI Backend Server and an Android Mobile App (PhoneClusterApp).
Together, they form a distributed storage network that repurposes old smartphones as storage nodes.

1. **Android App Module (PhoneClusterApp/app/src/main/java/com/phonecluster/app/)**

Dual-mode Android client — operates as either a User Node (upload/search) or a Storage Node (serve chunks).

File	Responsibility
FileChunker.kt	Splits selected files into fixed-size byte blocks and generates chunk metadata
AsconFileCrypto.kt	Encrypts and decrypts chunk data using ASCON-AEAD. Keys never leave the device
StorageNodeService.kt	Android Foreground Service — keeps the storage node alive in the background, manages heartbeat and WebSocket lifecycle
HeartbeatManager.kt	Sends REST heartbeat pings every 10 seconds to advertise node availability

File	Responsibility
SearchScreen.kt	Chat-style semantic search UI — vectorizes query, ranks files by similarity, returns top matches
WebSocketManager.kt	Maintains persistent outbound WebSocket to receive commands (DOWNLOAD_CHUNK, PUSH_CHUNK) from server
EmbeddingEngine.kt	Runs the all-MiniLM-L6-v2 ONNX model on-device to generate 384-dimensional sentence embeddings
SummaryEngine.kt	Extracts key sentences from file text using TF-IDF before embedding
AppDatabase.kt	Room DB singleton (files_db) — stores file metadata and embedding BLOBs locally

2. **API Routes Module** (app/api/routes/)

Handles all HTTP endpoints exposed by the server.

File	Responsibility
device.py	Device registration, listing, status updates, and cluster status (storage info)
files.py	File download (reassembles chunks from nodes) and file deletion (cascading cleanup)
dashboard.py	Chunk upload/download — phones push chunks here and pull chunks from here
chunks.py	Aggregated metrics for the monitoring dashboard (device stats, replication health, upload activity, storage distribution)

3. **WebSocket Module** (app/api/ws/)

File	Responsibility
devices.py	Orchestrates sending a chunk to assigned storage nodes over WebSocket. Marks assignments as FAILED if the target device is offline

4. **Services Module** (app/services/)

Core business logic layer — handles chunk distribution, replication, repair, and fault tolerance.

File	Responsibility
heartbeat.py	Processes heartbeat pings from devices, updating their last-seen timestamp
distribute_chunk.py	Persistent WebSocket connection with each storage node. Handles: device registration via fingerprint, sending DOWNLOAD_CHUNK commands to phones, receiving CHUNK_STORED_SUCCESS / CHUNK_STORE_FAILED acks, and marking replicas as LOST when a device disconnects
plan_replication.py	Selects which online devices (in Cluster mode) should store a given chunk. Maintains a target replica count of 2
repair_manager.py	Periodically scans all chunks, identifies under-replicated ones (fewer than 2 active replicas), and triggers re-replication to healthy nodes
offline_detection.py	Background loop that detects devices with stale heartbeats and marks them offline

5. **Data Models** (app/models/)

File	Responsibility
User	user_id, email, password_hash
Device	device_id, device_fingerprint, status (ONLINE/OFFLINE), mode (User/Cluster), storage_capacity, available_storage, last_heartbeat
Chunk	chunk_id, file_id, chunk_index, chunk_hash, chunk_size
File	file_id, user_id, file_name, file_size, num_chunks
ChunkReplication	replication_id, chunk_id, device_id, replica_status (REPLICATING / ACTIVE / LOST / FAILED)

5. Core Infrastructure (app/core/)

File	Responsibility
database.py	SQLAlchemy engine and session management (MySQL connection)
connection_manager.py	Maintains a registry of active WebSocket connections per device. Provides connect(), disconnect(), and send_command() methods
scheduler.py	Runs background async loops: offline monitor (detects stale heartbeats) and repair loop (fixes under-replicated chunks)
constants.py	Server URL, temp chunk directory path, and other configuration

ALGORITHMS USED

• Ascon-AEAD128 Encryption Algorithm

Input: 128-bit Key K, 128-bit Nonce N, Associated Data A, Plaintext P

Output: Ciphertext C and Authentication Tag T

1. Start the algorithm by selecting a fixed initialization vector (IV) and create the internal state S by combining the IV, the secret key K, and the nonce N.
2. Apply the Ascon permutation with 12 rounds to the internal state so that the values of the key, nonce, and IV are thoroughly mixed.
3. After the permutation, XOR the key again with part of the internal state to strengthen the security of the initialization.
4. If associated data (A) is present, divide it into 128-bit blocks.
5. If the last associated data block is smaller than 128 bits, pad it to reach the required size.
6. For each associated data block, XOR the block with the first 128 bits of the internal state and then apply the Ascon permutation with 8 rounds to update the state.
7. After all associated data blocks are processed, perform domain separation by XORing a special marker bit into the internal state to distinguish associated data from plaintext processing.
8. Divide the plaintext (P) into 128-bit blocks.

imes

9.For each plaintext block, XOR the block with the first 128 bits of the internal state and output the result as a ciphertext block.

10.After producing each ciphertext block, update the internal state using the Ascon permutation with 8 rounds.

11.If the final plaintext block is shorter than 128 bits, pad it, XOR it with the state, and output only the required number of bits as the last ciphertext block.

12.Combine all ciphertext blocks together to produce the complete ciphertext C.

13.In the finalization step, mix the secret key with the internal state again.

14.Apply the Ascon permutation with 12 rounds to the updated state.

15.Generate the authentication tag (T) from part of the internal state combined with the key.

16.Finally, output the pair (C, T), where C is the ciphertext and T is the authentication tag used to verify the integrity and authenticity of the message.

• Ascon-AEAD128 Decryption Algorithm

Input: 128-bit Key K, 128-bit Nonce N, Associated Data A, Ciphertext C, Authentication Tag T

Output: Plaintext P or Fail

1. Start by selecting the fixed initialization vector (IV) and create the internal state S by combining the IV, the secret key K, and the nonce N.
2. Apply the Ascon permutation with 12 rounds to the internal state so that the key, nonce, and IV are properly mixed.
3. XOR the key again with part of the internal state to complete the initialization process.
4. If associated data (A) exists, divide it into 128-bit blocks.
5. If the final associated data block is smaller than 128 bits, pad it to reach the required block size.
6. For each associated data block, XOR the block with the first 128 bits of the internal state and apply the Ascon permutation with 8 rounds to update the state.
7. After all associated data blocks are processed, perform domain separation by XORing a special marker bit into the internal state to distinguish associated data processing from ciphertext processing.
8. Divide the ciphertext (C) into 128-bit blocks.

9.For each ciphertext block (except the last one), recover the plaintext block by XORing the ciphertext block with the first 128 bits of the internal state.

10.Update the internal state by replacing its first 128 bits with the ciphertext block and then apply the Ascon permutation with 8 rounds.

11.For the final ciphertext block, determine its length and recover the remaining plaintext bits by XORing it with the corresponding bits of the internal state.

12.Update the internal state using padding and by inserting the final ciphertext block into the state as specified by the algorithm.

13.In the finalization step, mix the secret key into the internal state again.

14.Apply the Ascon permutation with 12 rounds to the updated state.

15.Generate a new authentication tag T′ from part of the internal state combined with the key.

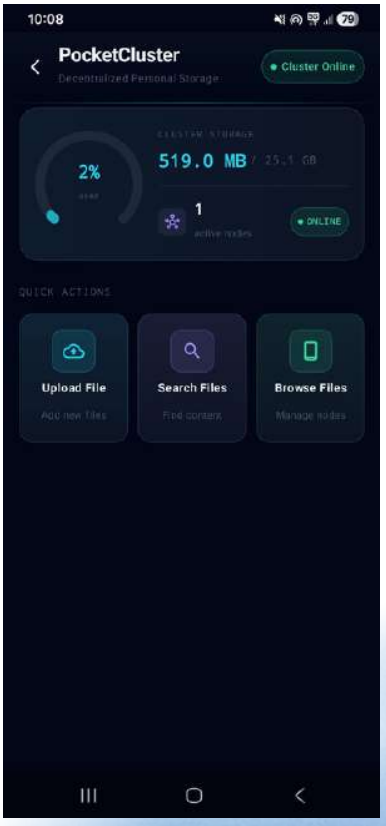
16.Compare the generated tag T′ with the received tag T.

17.If the two tags match, combine all recovered plaintext blocks to produce the complete plaintext P and return it.

18.If the tags do not match, return fail, indicating that the ciphertext or associated data has been modified or is invalid.

RESULTS

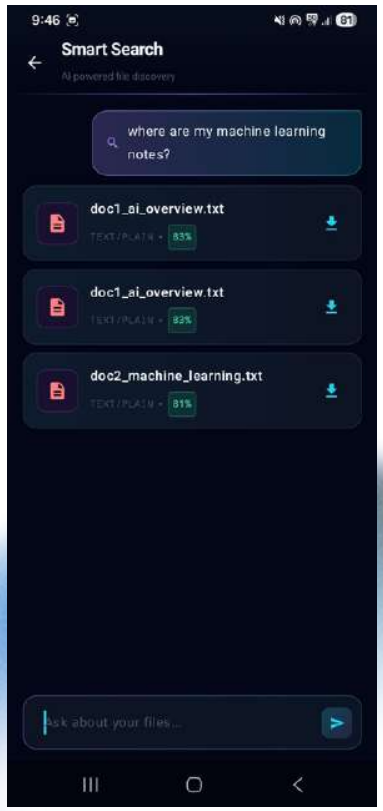
9:45
PocketCluster
Decentralized personal storage
Device #4
User Mode
Upload, browse & manage your files
Storage Mode
Contribute storage space to the cluster
You can switch modes anytime from settings


10:08
PocketCluster
Decentralized Personal Storage
Cluster Online
2%
519.0 MB
1
ONLINE
QUICK ACTIONS
Upload File
Search Files
Browse Files

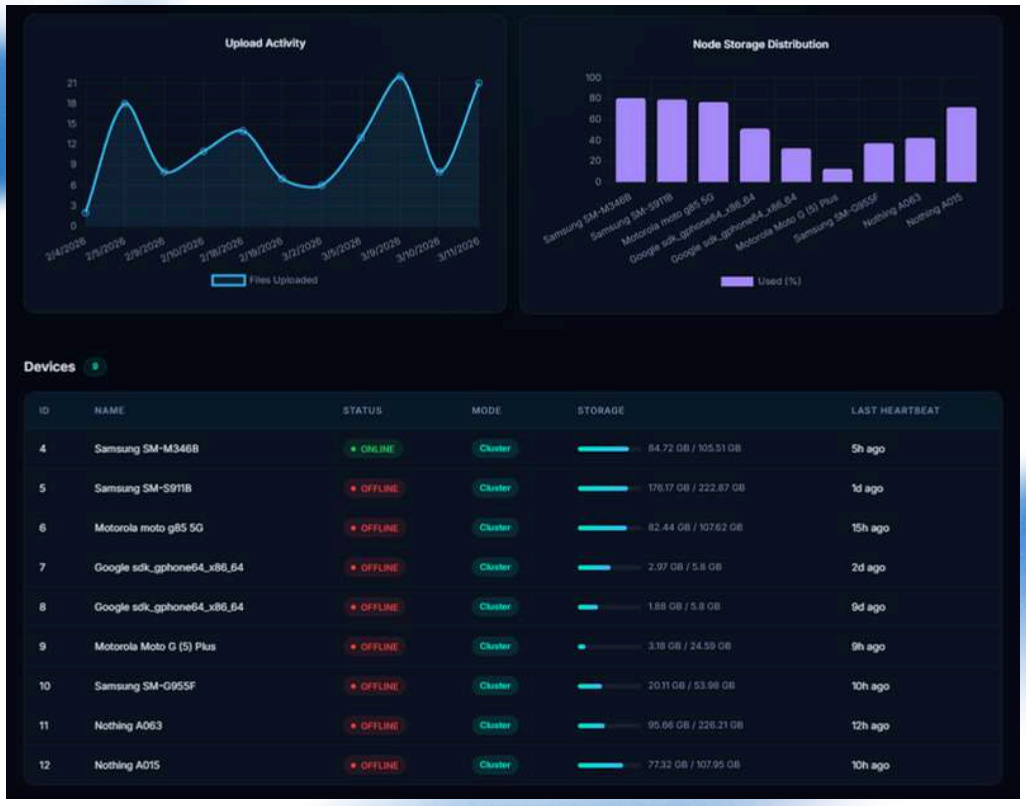
9:47
#4
NODE
Storage Node Active
Serving as a storage node in the PocketCluster network
ONLINE
STORAGE
P2P
Activity
Waiting for chunk assignments...
Heartbeat: connected to coordinator

10:01
Quick Actions
Upload File
Search Files
Browse Files
Selected File
Team2_CG_Kevin_Jose_57.pdf
28.7 MB · application/pdf
3
10 KB
28.7 MB
Prepare Chunks
Upload to Cluster
CHUNKS 3
Chunk #0
Ready
Chunk #1
Ready
Chunk #2
Ready

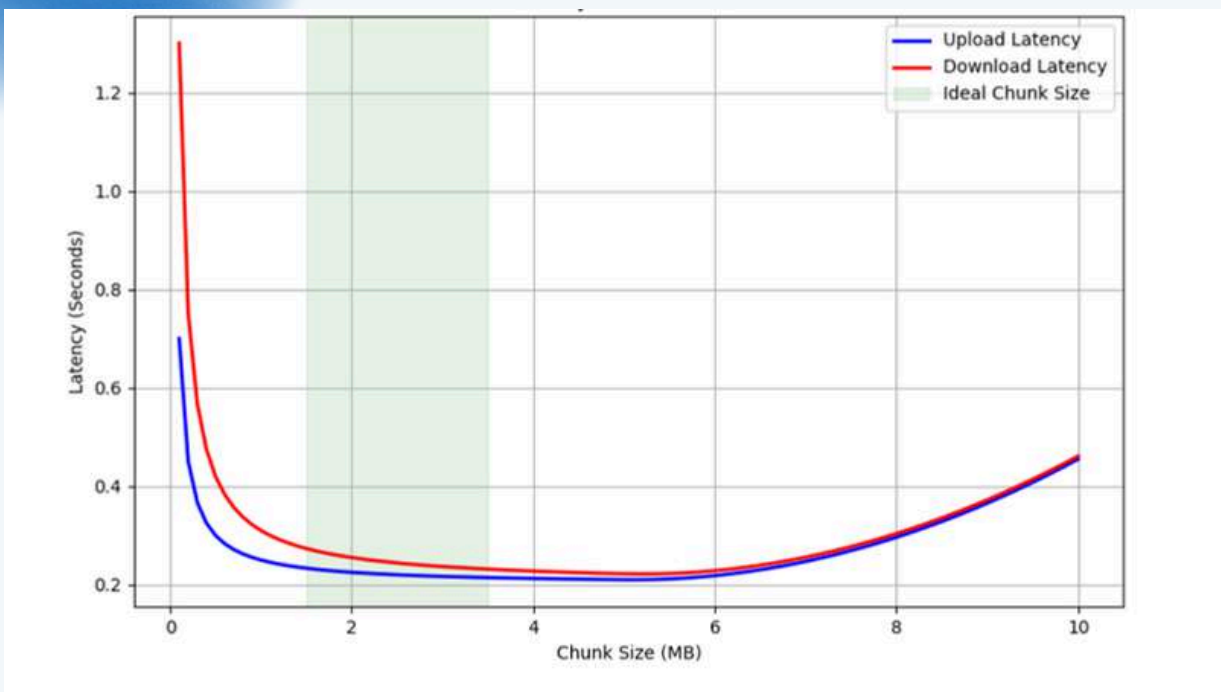
10:13
My Files
5.0 MB
16
3
ai_future.txt
DOCUMENT · 622 B
blockchain_technology.txt
DOCUMENT · 490 B
climate_change_overview.txt
DOCUMENT · 495 B
cybersecurity_basics.txt
DOCUMENT · 473 B
data_science_workflow.txt
DOCUMENT · 521 B
doc1_ai_overview.txt


9:46
Smart Search
where are my machine learning notes?
doc1_ai_overview.txt
doc1_ai_overview.txt
doc2_machine_learning.txt
Ask about your files...

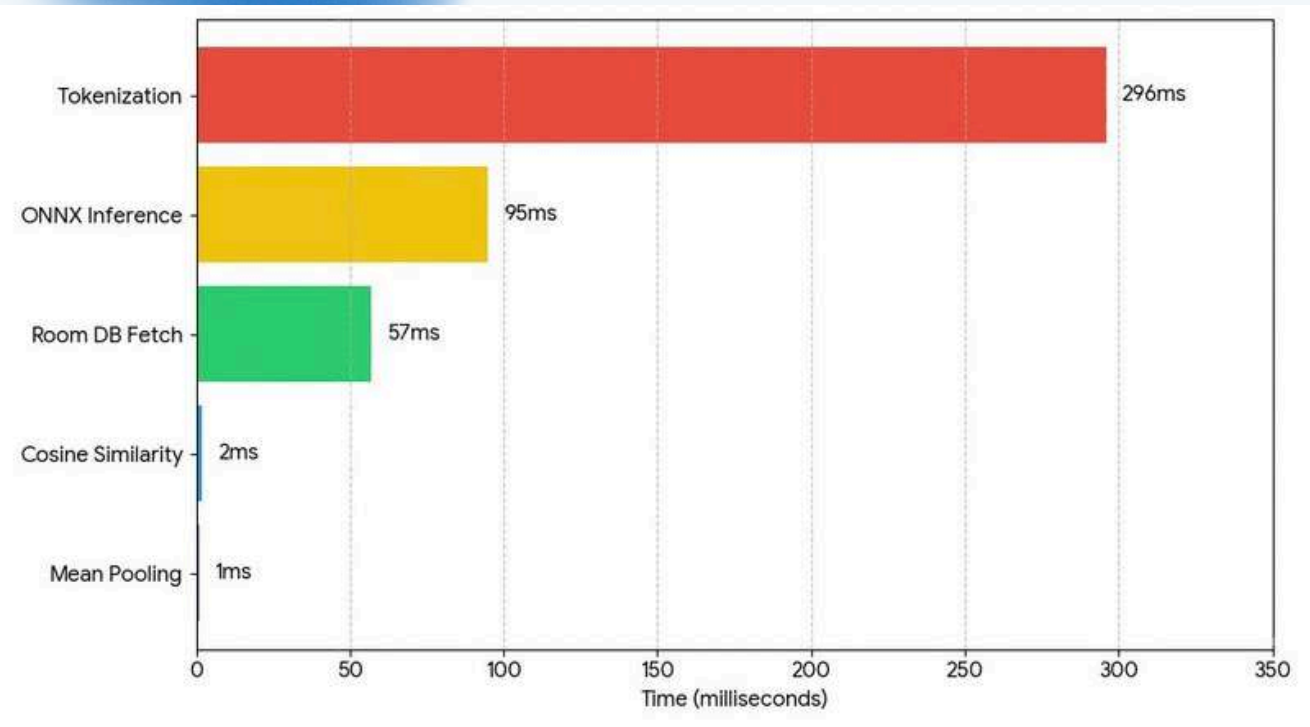
Cluster Overview
1
Total Users
9
Total Devices
1
Online Devices
8
Offline Devices
130
Total Files
251
Total Replication Chunks
Storage Utilization
Cluster Storage Capacity
527.07 MB / 105.51 GB
Cluster Analytics
Cluster Storage Usage
Used
Free
Device Status
Online
Offline
Replication Health
Active
Replicating
Lost
Failed



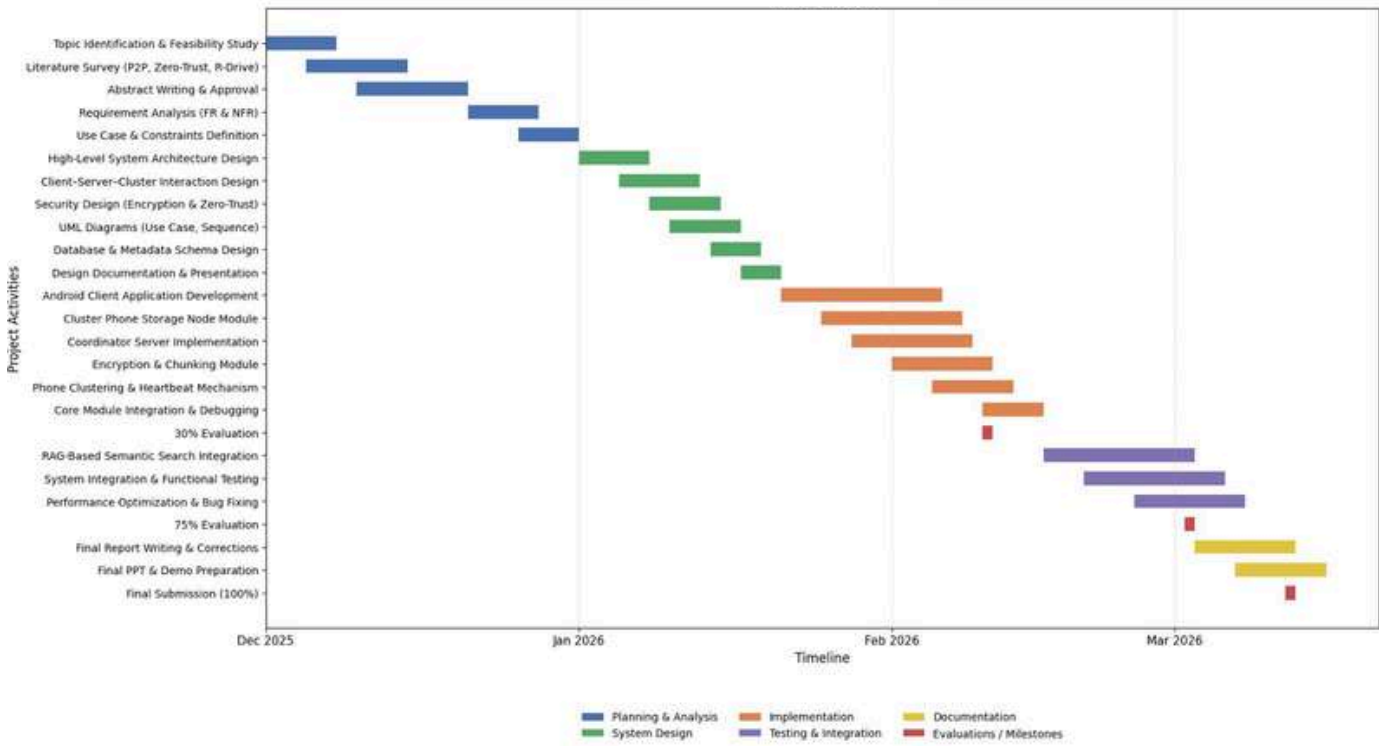
Latency vs Chunk Size



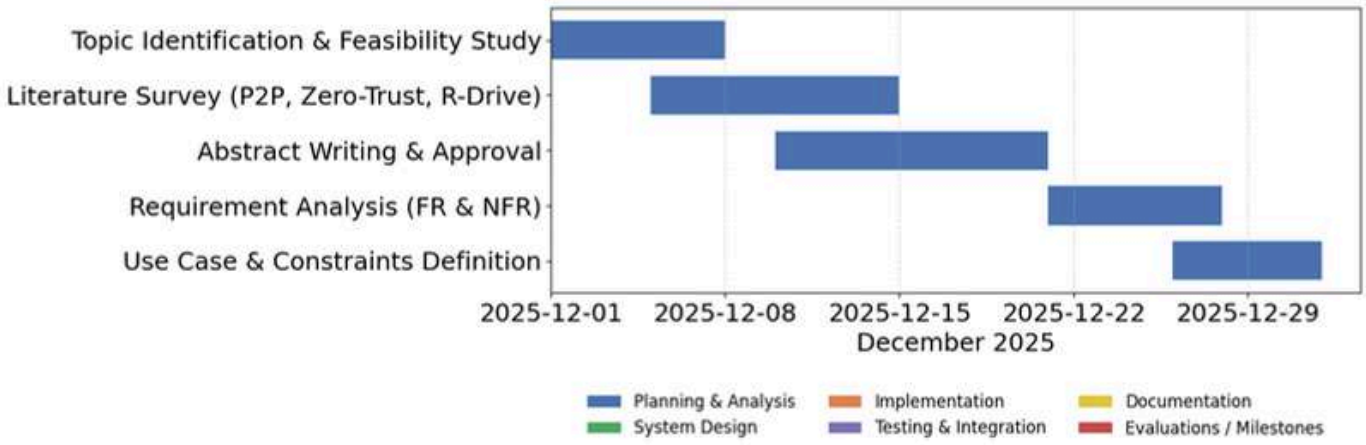
Semantic Search Latency Breakdown



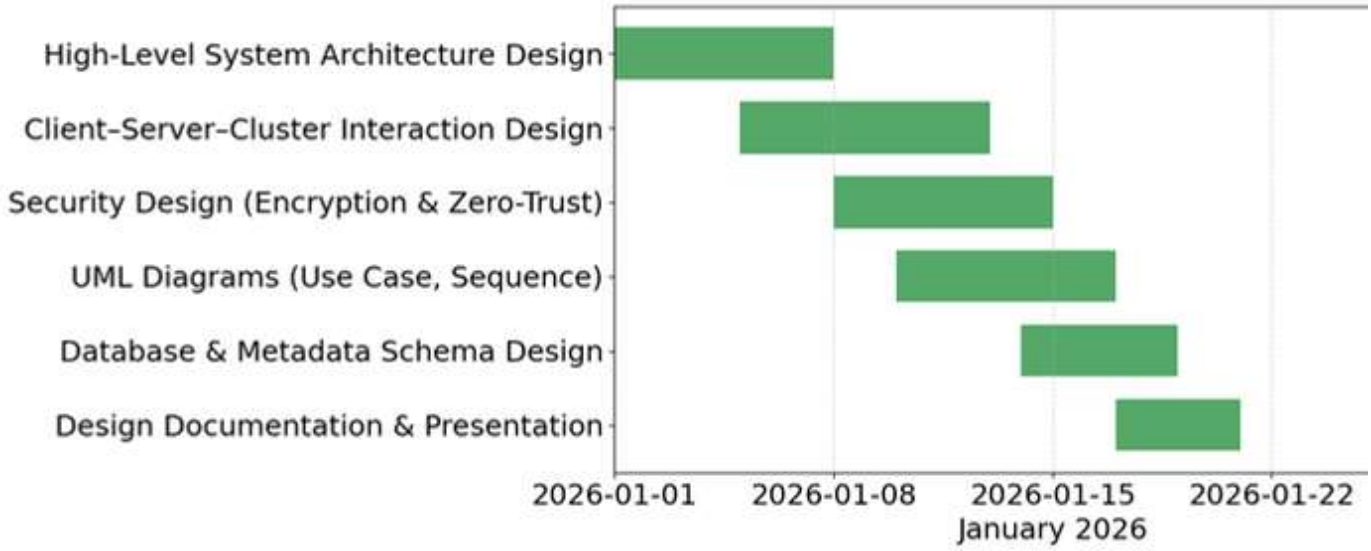
GANTT CHART



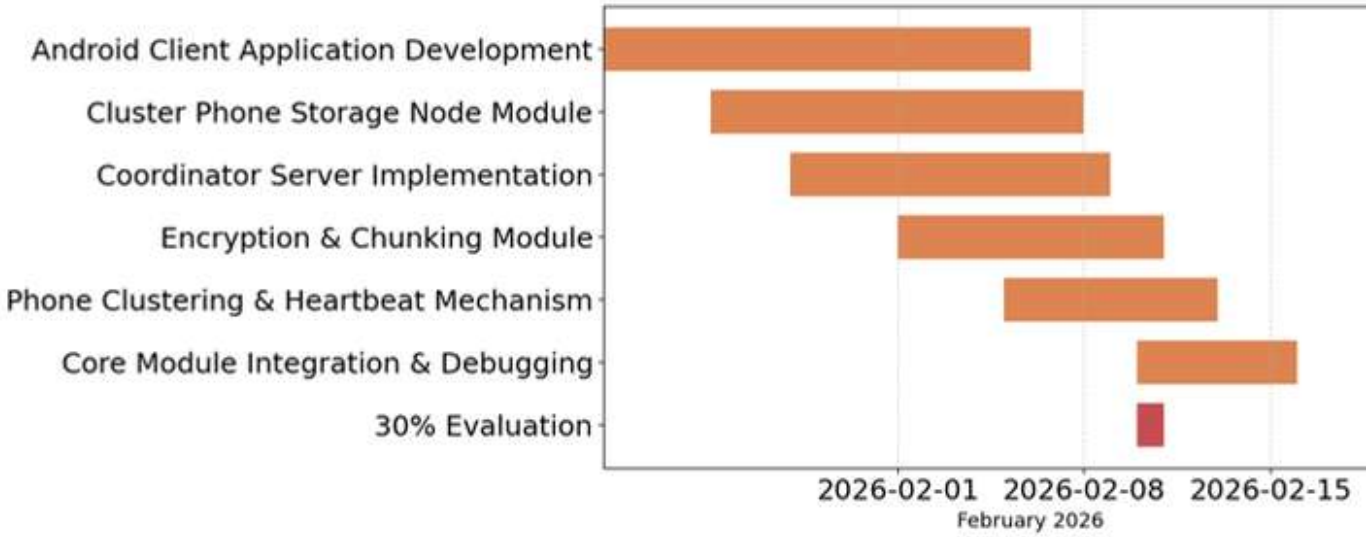
Gantt Chart - **December**



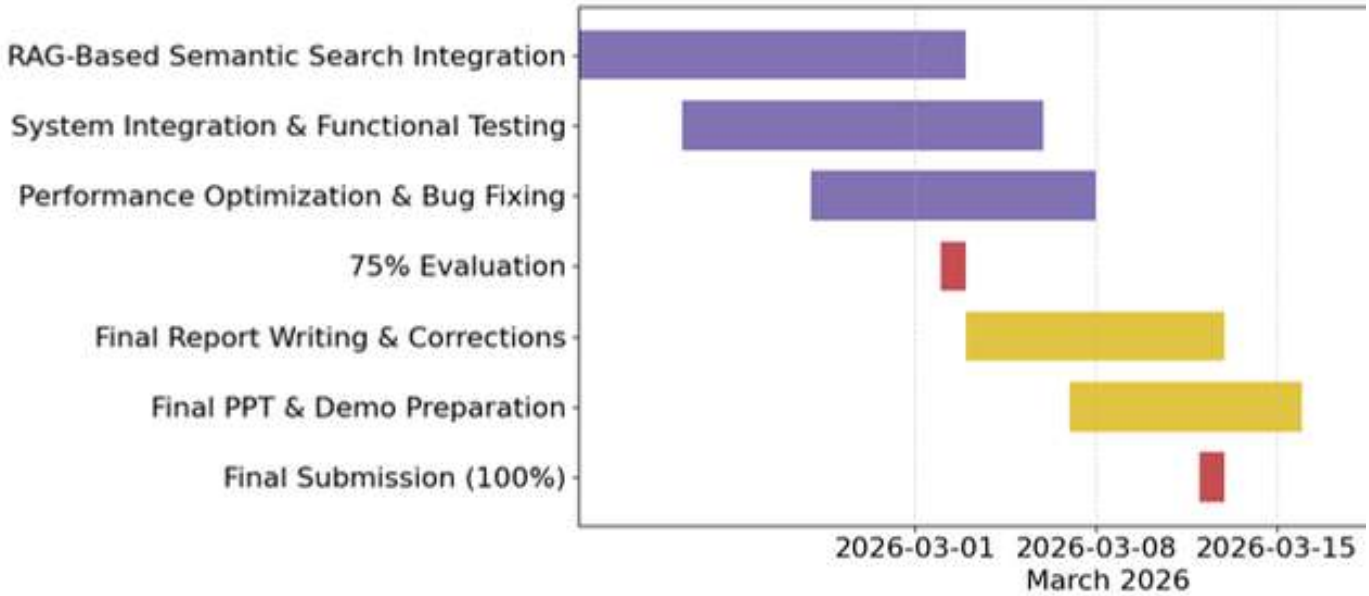
Gantt Chart - **January**



Gantt Chart - **February**



Gantt Chart - **March**



THANK YOU

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

RAJAGIRI SCHOOL OF ENGINEERING & TECHNOLOGY (AUTONOMOUS)

RAJAGIRI VALLEY, KAKKANAD, KOCHI, 682039

(Affiliated to APJ Abdul Kalam Technological University)

Vision, Mission, Programme Outcomes and Course Outcomes

Institute Vision

To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

Institute Mission

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Department Vision

To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

Department Mission

To inspire and nurture students, with up-to-date knowledge in Computer Science and Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

Programme Outcomes (PO)

Engineering Graduates will be able to:

PO1: Engineering Knowledge

Apply knowledge of mathematics, computing, networking, and engineering fundamentals to develop a decentralized mobile cloud storage system using Android (Kotlin), FastAPI, WebSockets, database management (MySQL), and cryptographic techniques such as encryption and hashing.

PO2: Problem Analysis

Identify and analyze the limitations of centralized cloud storage systems such as dependency on third-party providers, privacy risks, and lack of user control, and formulate a decentralized solution based on user-owned mobile devices with secure data handling and efficient resource utilization.

PO3: Design/Development of Solutions

Design and develop a secure, user-managed distributed storage system that uses client-side encryption, chunk-based file storage, replication strategies, and real-time coordination between devices, ensuring reliability, data integrity, and efficient storage utilization while maintaining user privacy.

PO4: Conduct Investigations of Complex Problems

Conduct investigations on system performance, including device availability (heartbeat mechanism), chunk replication reliability, failure handling, and data integrity verification using hashing, and analyze results to improve system robustness and fault tolerance.

PO5: Engineering Tool Usage

Select and apply modern engineering tools and technologies such as FastAPI (backend), WebSockets (real-time communication), Android foreground services, SQLAlchemy ORM, MySQL database, and ONNX-based machine learning models for semantic search, while understanding their limitations in distributed environments.

PO6: The Engineer and The World

Evaluate the societal and environmental impact of decentralized storage by reducing reliance on large data centers, promoting user data ownership, and ensuring privacy through zero-trust architecture, while considering sustainability and ethical data usage.

PO7: Ethics
Apply ethical principles by ensuring data privacy through end-to-end encryption, preventing unauthorized access via metadata-based access control, and maintaining a zero-trust system where neither the server nor storage nodes can access user data.

PO8: Individual and Collaborative Team Work
Work effectively as a team to design and implement different system components such as backend services, Android application development, encryption mechanisms, and UI/UX design, while coordinating integration and testing across modules.

PO9: Communication
Communicate effectively through technical documentation, structured project logs, API design explanations, and presentations, clearly conveying system architecture, workflows, and implementation details to both technical and non-technical audiences.

PO10: Project Management and Finance
Apply project management principles in designing a cost-efficient system that eliminates dependency on paid cloud storage services by leveraging user-owned devices, and manage development tasks such as module division, testing, and deployment.

PO11: Life-Long Learning
Demonstrate adaptability by learning and applying new technologies such as ONNX models, Android system services, WebSocket communication, and distributed system design concepts, while continuously improving the system based on testing and feedback.

Programme Specific Outcomes (PSO)
A graduate of the Computer Science and Engineering Program will demonstrate:

PSO1: Computer Science Specific Skills
Ability to design and analyze decentralized distributed systems using core concepts of networking, databases, operating systems, and cryptography.

PSO2: Programming and Software Development Skills
Ability to develop scalable applications using modern frameworks such as Android (Kotlin), FastAPI, and implement real-time communication using WebSockets and REST APIs.

PSO3: Professional Skills
Ability to develop innovative, privacy-focused software solutions such as decentralized storage systems addressing real-world societal needs.

Course Outcomes (CO)

CO1: Analyze the limitations of centralized cloud storage systems and design a decentralized mobile-based storage solution using distributed architecture, encryption, and chunk-based mechanisms.

CO2: Investigate and implement secure and privacy-preserving techniques such as client-side encryption, zero-trust architecture, and user-controlled storage using mobile devices.

CO3: Design and develop an integrated system consisting of Android applications and backend services using FastAPI, WebSockets, and databases for distributed storage coordination.

CO4: Implement and evaluate system workflows including device registration, heartbeat monitoring, chunk replication, failure handling, and file retrieval ensuring data integrity and system reliability.

CO5: Communicate system design, evaluate performance, and propose improvements in terms of fault tolerance, replication strategies, and decentralized storage efficiency through technical documentation and presentations.

Appendix C: CO-PO-PSO Mapping

COURSE OUTCOMES (CO)

SNO	Description	Bloom's Taxonomy Level
1	Analyze the limitations of centralized cloud storage systems and design a decentralized mobile-based storage solution using distributed architecture, encryption, and chunk-based mechanisms.	Apply
2	Investigate and implement secure and privacy-preserving techniques such as client-side encryption, zero-trust architecture, and user-controlled storage using mobile devices.	Apply
3	Design and develop an integrated system consisting of Android applications and backend services using FastAPI, WebSockets, and databases for distributed storage coordination.	Apply
4	Implement and evaluate system workflows including device registration, heartbeat monitoring, chunk replication, failure handling, and file retrieval ensuring data integrity and system reliability.	Apply
5	Communicate system design, evaluate performance, and propose improvements in terms of fault tolerance, replication strategies, and decentralized storage efficiency through technical documentation and presentations.	Apply

MAPPING COURSE OUTCOMES (COs) – PROGRAM OUTCOMES (POs) AND PROGRAM SPECIFIC OUTCOMES (PSOs)

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	3	3	3	2	2	2	2	1	1	1	2	3	3	3
CO2	2	3	3	1	2	3	3	1	1	1	2	2	3	3
CO3	3	2	3	1	3	1	1	3	2	2	2	3	3	3
CO4	2	2	3	3	3	2	3	2	2	3	2	2	3	3
CO5	2	3	3	3	2	2	2	2	3	2	3	2	2	3

(1 – Low, 2 – Medium, 3 – High)

JUSTIFICATIONS FOR CO-PO MAPPING

Mapping	Level	Justification
101003/CS622T.CO1-PO1	High	Applies engineering knowledge of distributed systems, networking, and mobile computing to design a decentralized storage system using Android devices and backend coordination.
101003/CS622T.CO1-PO2	High	Analyzes limitations of centralized cloud systems such as dependency, privacy risks, and lack of user control.
101003/CS622T.CO1-PO3	High	Designs a distributed architecture with chunk-based storage, replication, and secure retrieval.
101003/CS622T.CO1-PO4	Medium	Includes validation of chunk integrity, replication reliability, and system availability.
101003/CS622T.CO1-PO5	Medium	Uses modern tools such as FastAPI, WebSockets, and Android frameworks.
101003/CS622T.CO1-PO6	Medium	Considers decentralized storage as a sustainable alternative.
101003/CS622T.CO1-PO7	Medium	Ensures ethical data handling through encryption and zero-trust principles.
101003/CS622T.CO1-PO8	Low	Requires basic collaboration among team members during system design.
101003/CS622T.CO1-PO9	Low	Involves documentation and communication of system architecture.
101003/CS622T.CO1-PO10	Low	Includes planning and execution of system components.
101003/CS622T.CO1-PO11	Medium	Encourages learning of distributed and mobile computing concepts.
101003/CS622T.CO1-PSO1	High	Applies computer science fundamentals in distributed system design.
101003/CS622T.CO1-PSO2	High	Implements programming techniques for chunking and communication.
101003/CS622T.CO1-PSO3	High	Develops innovative decentralized storage solutions.
101003/CS622T.CO2-PO1	Medium	Applies engineering knowledge for implementing secure and privacy-preserving storage mechanisms.

Mapping	Level	Justification
101003/CS622T.CO2-PO2	High	Analyzes privacy risks and security challenges in centralized systems.
101003/CS622T.CO2-PO3	High	Designs secure storage using encryption and zero-trust architecture.
101003/CS622T.CO2-PO4	Low	Limited validation of security mechanisms during implementation.
101003/CS622T.CO2-PO5	Medium	Uses modern frameworks for implementing secure systems.
101003/CS622T.CO2-PO6	High	Emphasizes societal benefits like privacy and user data ownership.
101003/CS622T.CO2-PO7	High	Ensures ethical responsibility through secure data handling.
101003/CS622T.CO2-PO8	Low	Requires minimal collaboration during security implementation.
101003/CS622T.CO2-PO9	Low	Involves communication of security features.
101003/CS622T.CO2-PO10	Low	Includes structured implementation of secure workflows.
101003/CS622T.CO2-PO11	Medium	Encourages learning of cryptographic techniques.
101003/CS622T.CO2-PSO1	Medium	Applies computer science fundamentals for secure system design.
101003/CS622T.CO2-PSO2	High	Implements encryption and secure communication.
101003/CS622T.CO2-PSO3	High	Supports development of innovative privacy-focused systems.
101003/CS622T.CO3-PO1	High	Applies engineering knowledge to develop Android and backend systems.
101003/CS622T.CO3-PO2	Medium	Involves analysis of system requirements and integration challenges.
101003/CS622T.CO3-PO3	High	Designs and develops integrated distributed storage systems.
101003/CS622T.CO3-PO4	Low	Limited experimental validation during development.

Mapping	Level	Justification
101003/CS622T.CO3-PO5	High	Uses tools such as FastAPI, WebSockets, and databases.
101003/CS622T.CO3-PO6	Low	Minimal societal considerations during development.
101003/CS622T.CO3-PO7	Low	Basic ethical considerations during development.
101003/CS622T.CO3-PO8	High	Requires teamwork for frontend and backend integration.
101003/CS622T.CO3-PO9	Medium	Involves communication during system integration.
101003/CS622T.CO3-PO10	Medium	Requires coordination and project management.
101003/CS622T.CO3-PO11	Medium	Encourages continuous learning during development.
101003/CS622T.CO3-PSO1	Medium	Applies system design principles.
101003/CS622T.CO3-PSO2	High	Uses programming skills for implementation.
101003/CS622T.CO3-PSO3	High	Supports collaborative development of innovative systems.
101003/CS622T.CO4-PO1	Medium	Applies engineering knowledge in implementing system workflows.
101003/CS622T.CO4-PO2	Medium	Requires analysis of workflows like replication and retrieval mechanisms.
101003/CS622T.CO4-PO3	High	Implements workflows like chunking, replication, and retrieval.
101003/CS622T.CO4-PO4	High	Evaluates system performance through monitoring and failure handling.
101003/CS622T.CO4-PO5	High	Uses modern tools including Android services and backend frameworks.
101003/CS622T.CO4-PO6	Medium	Considers usability and deployment aspects.
101003/CS622T.CO4-PO7	High	Ensures secure and ethical implementation.

Mapping	Level	Justification
101003/CS622T.CO4-PO8	Medium	Requires coordination among team members and modules.
101003/CS622T.CO4-PO9	Medium	Involves documentation of workflows and implementation.
101003/CS622T.CO4-PO10	High	Involves planning and managing development processes.
101003/CS622T.CO4-PO11	Medium	Encourages debugging and system improvement.
101003/CS622T.CO4-PSO1	Medium	Applies system design principles in implementation.
101003/CS622T.CO4-PSO2	High	Applies software engineering practices.
101003/CS622T.CO4-PSO3	Medium	Uses computer science concepts in execution.
101003/CS622T.CO5-PO1	Medium	Applies technical knowledge to evaluate system performance.
101003/CS622T.CO5-PO2	High	Evaluates centralized systems and identifies improvements.
101003/CS622T.CO5-PO3	High	Proposes improvements in replication and fault tolerance.
101003/CS622T.CO5-PO4	High	Analyzes system reliability and performance.
101003/CS622T.CO5-PO5	Medium	Uses tools and techniques for system evaluation.
101003/CS622T.CO5-PO6	Medium	Considers societal impact such as privacy and decentralization.
101003/CS622T.CO5-PO7	Medium	Ensures ethical evaluation of system improvements.
101003/CS622T.CO5-PO8	Low	Requires minimal teamwork during evaluation phase.
101003/CS622T.CO5-PO9	High	Communicates system design and evaluation effectively.
101003/CS622T.CO5-PO10	Medium	Involves structured documentation and planning.

Mapping	Level	Justification
101003/CS622T.CO5-PO11	High	Encourages continuous learning and improvement.
101003/CS622T.CO5-PSO1	Medium	Applies analytical skills to evaluate systems.
101003/CS622T.CO5-PSO2	Medium	Uses programming practices for system improvement.
101003/CS622T.CO5-PSO3	High	Supports communication of innovative solutions.